

The Package’s Python Surface

Every module, class, function and public method, with the summary line it carries in its own docstring

M. J. Wilson

January 1, 2025

Abstract

A map of what the package exposes, generated from the sources rather than written: ‘infra/api_map.py’ reads each module by ‘ast’ — never by importing it, so no import side effect and no compiled extension can drop a module from the map — and lays out the first paragraph of every docstring it finds. The companion documents state the mathematics (the textbook) and the measurements (the paper); this one states the surface, and it is the only one of the three whose content no one edits.

Contents

1	How to read this map	6
2	The package root	6
2.1	__init__	6
2.2	backend	6
2.3	bound	6
2.4	emissions	6
2.5	enumeration	11
2.6	fixtures	12
2.7	incidence	12
2.8	inputs	12
2.9	log	12
2.10	numerics	13
2.11	parallel	13
2.12	ragged	13
2.13	track	13
3	learn/	14
3.1	learn/__init__	14
3.2	learn/actor_critic	14
3.3	learn/arena	15
3.4	learn/canonical	15
3.5	learn/critic	16
3.6	learn/environment	17
3.7	learn/exact	17
3.8	learn/failure	17
3.9	learn/gym	18
3.10	learn/hmm	18
3.11	learn/keyed	18

3.12	learn/planning	19
3.13	learn/policy	19
3.14	learn/potts	20
3.15	learn/potts_nd	21
3.16	learn/ppo	22
3.17	learn/ranking	22
3.18	learn/reinforce	22
3.19	learn/relaxed	23
3.20	learn/rollout	23
3.21	learn/surrogate	23
3.22	learn/tabular	25
3.23	learn/tree	25
4	likelihood/	26
4.1	likelihood/__init__	26
4.2	likelihood/algebraic	26
4.3	likelihood/belief_propagation	26
4.4	likelihood/blocks	26
4.5	likelihood/brute_force	27
4.6	likelihood/convolutional	27
4.7	likelihood/convolutional_rust	27
4.8	likelihood/css	27
4.9	likelihood/device	28
4.10	likelihood/distance	28
4.11	likelihood/features	28
4.12	likelihood/forward_backward	29
4.13	likelihood/hadamard	29
4.14	likelihood/hmm_paths	29
4.15	likelihood/ldpc	30
4.16	likelihood/message_passing	30
4.17	likelihood/message_passing_reference	30
4.18	likelihood/message_passing_rust	31
4.19	likelihood/mixture_assignments	31
4.20	likelihood/objective	31
4.21	likelihood/parsimony	32
4.22	likelihood/patterns	32
4.23	likelihood/polar	32
4.24	likelihood/potts	33
4.25	likelihood/pruning	33
4.26	likelihood/pruning_analytic	33
4.27	likelihood/pruning_common	33
4.28	likelihood/pruning_rust	34
4.29	likelihood/pruning_torch	34
4.30	likelihood/ragged_rust	34
4.31	likelihood/schedule	34
4.32	likelihood/spatio_sequential	37
4.33	likelihood/spatio_sequential_rust	37
4.34	likelihood/surrogate	37
4.35	likelihood/turbo	38

5	opt/	39
5.1	opt/__init__	39
5.2	opt/budget	39
5.3	opt/constrain	39
5.4	opt/em	40
5.5	opt/emission_mixture	40
5.6	opt/fit	40
5.7	opt/hmm	40
5.8	opt/initialize	42
5.9	opt/mixture	42
5.10	opt/objective	43
5.11	opt/potts	43
5.12	opt/testfunctions	44
6	qa/	45
6.1	qa/__init__	45
6.2	qa/backend_agreement	45
6.3	qa/build	45
6.4	qa/coupled_labelling	45
6.5	qa/figure	46
6.6	qa/frustrated_lattices	46
6.7	qa/hmc_warmup	46
6.8	qa/layout	47
6.9	qa/likelihood_footprint	47
6.10	qa/manifest	47
6.11	qa/mixture_seeding	48
6.12	qa/opt_branch_recovery	48
6.13	qa/opt_coverage	48
6.14	qa/opt_model_recovery	48
6.15	qa/opt_recovery	48
6.16	qa/optimizer_landscapes	49
6.17	qa/parsimony_zones	49
6.18	qa/rl_reward_surface	49
6.19	qa/rl_tree_policy	49
6.20	qa/runner	49
6.21	qa/search_topologies	50
6.22	qa/search_trajectory	50
6.23	qa/sim_example	50
6.24	qa/sim_problem_sizes	50
6.25	qa/sim_tree	50
6.26	qa/style	51
6.27	qa/tanner_graph	51
6.28	qa/topology_accuracy	51
6.29	qa/tropical_relaxation	51
6.30	qa/turbo_waterfall	52
7	sample/	52
7.1	sample/__init__	52
7.2	sample/accept	52
7.3	sample/annealed	52
7.4	sample/balanced	53
7.5	sample/gibbs	53

7.6	sample/hmc	54
7.7	sample/initialize	54
7.8	sample/kernels	55
7.9	sample/langevin	55
7.10	sample/potts_keyed	55
7.11	sample/potts_mcmc	56
7.12	sample/schedule	56
7.13	sample/slice	57
7.14	sample/statistics	57
7.15	sample/tempered	57
8	sandbox/	58
8.1	sandbox/__init__	58
8.2	sandbox/circulant_schedule	58
8.3	sandbox/count_emissions	58
8.4	sandbox/maxflow_declined	60
8.5	sandbox/polar_reference	60
8.6	sandbox/pruning_burn	60
8.7	sandbox/rectangular_hmm	60
8.8	sandbox/region_graph	60
8.9	sandbox/tropical	61
9	scripts/	61
9.1	scripts/__init__	61
9.2	scripts/run_snakes_and_ladders	61
10	search/	62
10.1	search/__init__	62
10.2	search/alpha_expansion	62
10.3	search/bifurcation	62
10.4	search/decoding	62
10.5	search/ground_state	62
10.6	search/infer	63
10.7	search/initialize	63
10.8	search/max_cut	64
10.9	search/maxflow	64
10.10	search/maxflow_rust	64
10.11	search/neighbor_joining	65
10.12	search/projection	65
10.13	search/spatio_sequential	66
10.14	search/support	66
10.15	search/tightening	66
11	sim/	67
11.1	sim/__init__	67
11.2	sim/canonical	67
11.3	sim/capacity	67
11.4	sim/convolutional	67
11.5	sim/count_pairs	68
11.6	sim/count_pairs_rust	70
11.7	sim/css	70
11.8	sim/elementary_codes	70

11.9 sim/emission_mixture	71
11.10sim/factor_graph	71
11.11sim/fixtures	71
11.12sim/galois	72
11.13sim/graph	73
11.14sim/gtr	73
11.15sim/hmm	73
11.16sim/jc	74
11.17sim/ldpc	74
11.18sim/mixture	75
11.19sim/newick	76
11.20sim/params	76
11.21sim/polar	76
11.22sim/potts	77
11.23sim/potts_chain	77
11.24sim/reed_solomon	77
11.25sim/simulate	78
11.26sim/simulator	78
11.27sim/spatio_sequential	78
11.28sim/topology	79
11.29sim/tree	79

1 How to read this map

The package declares 183 modules, 313 public classes, 710 public top-level functions and 675 public methods: 1881 entries. Each is listed under the module that declares it, with the first paragraph of its own docstring. A path is relative to `python/snakes_and_ladders/`, so the entry `load_hmm_params` under `sim/hmm` is `sim/hmm/load_hmm_params`; a method is written under its class, qualified by it.

The order is the package layout and not the alphabet — the root’s modules, then each subpackage, and within a module the order the source declares — because the structure is part of what is being shown.

What is not here. Every underscore-prefixed name is excluded, and with an underscore-prefixed class go its methods: a private class’s members are no more public than the class. So are `typing.overload` stubs, which carry no docstring by convention and would be listed as missing one. Nothing else is filtered, and nothing is truncated.

91 entries carry no docstring and are marked as such, none of them a public top-level function — the generator fails rather than typeset one. 46 summaries exceed 100 characters and are wrapped by the layout rather than cut; the generator lists them so they can be shortened where they are written.

2 The package root

2.1 `__init__`

The package’s surface: the seams it declares, reachable from the root.
No public names.

2.2 `backend`

Which implementation runs a kernel, chosen by the caller and never silently.

<code>Backend</code>	<i>class.</i> Where a kernel runs.
----------------------	------------------------------------

2.3 `bound`

The seam between an exact evaluation and what stands in for it (issue #308).

<code>Bound</code>	<i>class.</i> What a surrogate claims about the exact value.
<code>Surrogate</code>	<i>class.</i> A stand-in for an exact evaluation, with its claim attached.
<code> Surrogate.kind</code>	<i>No docstring.</i>
<code>BoundViolation</code>	<i>class.</i> A surrogate claiming a bound was on the wrong side of the exact value.
<code>Certificate</code>	<i>class.</i> What certifying a surrogate over a set of structures records.
<code>certify</code>	Check a surrogate’s claim against the exact value on every structure.

2.4 `emissions`

What a hidden state emits, separated from how the model is fitted.

<code>Reestimate</code>	<i>class.</i> One emission M step, and whether what it returned is an answer.
-------------------------	---

CovariateNotSupportedError	<i>class.</i> A family was given a per-observation covariate it cannot condition on.
trial_count	The trial count to score at: the covariate's, or the declared one.
exposure	The exposure to scale the rate by, shaped to broadcast along the states.
validated_exposure	covariate as exposures, in <code>declared</code> 's dtype.
validated_trials	covariate as trial counts, in <code>declared</code> 's dtype.
refuse_covariate	Raise unless <code>covariate</code> is <code>None</code> .
CountEmissionFamily	<i>class.</i> An emission family over the non-negative integers, which has moments.
CountEmissionFamily.mean	Per-state mean, shape <code>(n_states,)</code> .
CountEmissionFamily.variance	Per-state variance, shape <code>(n_states,)</code> , or per channel.
EmissionFamily	<i>class.</i> The emission distribution of every hidden state, as one object.
EmissionFamily.n_states	Hidden states this family emits from.
EmissionFamily.is_discrete	Whether <code>log_density</code> returns a probability rather than a density.
EmissionFamily.observation_dtype	Type an observation is carried in: integral or floating point.
EmissionFamily.sample	Draw one observation per entry of <code>states</code> .
EmissionFamily.log_density	Score every observation under every state.
EmissionFamily.bregman_divergence	<code>D_phi(T(y), mu_state)</code> : the divergence of this family's log-partition.
EmissionFamily.validate	Raise if <code>observations</code> cannot have come from this family.
EmissionFamily.reestimate	The Baum-Welch M step for this family alone.
EmissionFamily.alignment_key	Per-state signature the hidden-state permutation is matched on.
EmissionFamily.named_parameters	This family's parameters under the names the model states them in.
CategoricalEmission	<i>class.</i> A symbol drawn from a per-state distribution over a fixed alphabet.
CategoricalEmission.from_log	Build from log-probabilities, the form the recursions carry.
CategoricalEmission.n_states	Hidden states this family emits from.
CategoricalEmission.n_symbols	Size of the emission alphabet.
CategoricalEmission.is_discrete	True: the alphabet is finite, so the evidence is a probability.
CategoricalEmission.observation_dtype	Symbols are indices into the alphabet.
CategoricalEmission.log_matrix	Log emission matrix, shape <code>(n_states, n_symbols)</code> .
CategoricalEmission.matrix	Emission matrix as probabilities, shape <code>(n_states, n_symbols)</code> .
CategoricalEmission.sample	Draw one symbol per entry of <code>states</code> by inverse-CDF sampling.

<code>CategoricalEmission.log_density</code>	Gather $\log B[:, \text{symbol}]$ for every observation.
<code>CategoricalEmission.bregman_divergence</code>	$-\log P(\text{symbol} \mid \text{state})$: the best member puts all its mass on the symbol.
<code>CategoricalEmission.validate</code>	Raise if a symbol lies outside the alphabet.
<code>CategoricalEmission.reestimate</code>	Normalized expected symbol counts, in log space.
<code>CategoricalEmission.alignment_key</code>	The emission rows themselves, as probabilities.
<code>CategoricalEmission.named_parameters</code>	<code>log_emission</code> , the form the forward recursion consumes.
<code>GaussianEmission</code>	<i>class.</i> A real observation drawn from <code>Normal(mean[state], scale[state])</code> .
<code>GaussianEmission.n_states</code>	Hidden states this family emits from.
<code>GaussianEmission.n_channels</code>	Entries an observation carries; 1 for the single-channel family.
<code>GaussianEmission.is_discrete</code>	False: the support is the real line, so the evidence is a density.
<code>GaussianEmission.observation_dtype</code>	Observations are real.
<code>GaussianEmission.mean</code>	Per-state mean, of the shape it was given.
<code>GaussianEmission.scale</code>	Per-state standard deviation, of <code>mean</code> 's shape.
<code>GaussianEmission.variance_floor</code>	Variance at or below which a re-estimate is refused.
<code>GaussianEmission.sample</code>	Draw one real observation per entry of <code>states</code> .
<code>GaussianEmission.log_density</code>	The Normal log-density of every observation under every state.
<code>GaussianEmission.bregman_divergence</code>	$((y - \mu) / \text{scale}) ** 2 / 2$, summed over the channels.
<code>GaussianEmission.validate</code>	Raise if an observation is not finite, or carries the wrong channels.
<code>GaussianEmission.reestimate</code>	Posterior-weighted mean and variance, in closed form.
<code>GaussianEmission.alignment_key</code>	The per-state means, one row per state and one column per channel.
<code>GaussianEmission.named_parameters</code>	<code>mean</code> and <code>scale</code> , the parameters the model is stated in.
<code>NegativeBinomialEmission</code>	<i>class.</i> A count drawn from <code>NegativeBinomial(dispersion[state], mean[state])</code> .
<code>NegativeBinomialEmission.from_probability</code>	Build from the textbook (r, p) , where $\mu = r (1 - p) / p$.
<code>NegativeBinomialEmission.n_states</code>	Hidden states this family emits from.
<code>NegativeBinomialEmission.is_discrete</code>	True: the support is the non-negative integers.
<code>NegativeBinomialEmission.observation_dtype</code>	Floating point, though the support is integral.
<code>NegativeBinomialEmission.dispersion</code>	Per-state <code>r</code> , shape <code>(n_states,)</code> .

NegativeBinomialEmission.	Per-state μ , shape $(n_states,)$.
mean	
NegativeBinomialEmission.	$\mu + \mu^2 / r$, the relation that makes this a count
variance	model.
NegativeBinomialEmission.	Draw one count per entry of <code>states</code> .
sample	
NegativeBinomialEmission.	The negative binomial log-probability of every count under
log_density	every state.
NegativeBinomialEmission.	$r \log((r + \mu) / (r + y)) + y \log(y (r + \mu) /$
bregman_divergence	$(\mu (r + y)))$.
NegativeBinomialEmission.	Raise if an observation is not a non-negative integer.
validate	
NegativeBinomialEmission.	The M step: a closed form for the mean, and a solve for the
reestimate	dispersion.
NegativeBinomialEmission.	The per-state mean <i>and variance</i> , both in observation
alignment_key	units.
NegativeBinomialEmission.	dispersion and mean, the parameters the model is stated
named_parameters	in.
PoissonEmission	<i>class</i> . A count drawn from <code>Poisson(mean[state])</code> : the
	equidispersed case.
PoissonEmission.n_states	Hidden states this family emits from.
PoissonEmission.is_	True: the support is the non-negative integers.
discrete	
PoissonEmission.	Floating point, since <code>lgamma</code> consumes the counts.
observation_dtype	
PoissonEmission.mean	Per-state rate, shape $(n_states,)$.
PoissonEmission.variance	Equal to the mean. The defining property, and the one
	tested.
PoissonEmission.sample	Draw one count per entry of <code>states</code> .
PoissonEmission.log_	$y \log(\lambda) - \lambda - \log(y!)$.
density	
PoissonEmission.bregman_	$y \log(y / \lambda) - y + \lambda$, the generalized I-
divergence	divergence.
PoissonEmission.validate	Raise if an observation is not a non-negative integer.
PoissonEmission.	The posterior-weighted mean, in closed form.
reestimate	
PoissonEmission.	The per-state rate, as a column: it is the whole family.
alignment_key	
PoissonEmission.named_	mean, the only parameter there is.
parameters	
BinomialEmission	<i>class</i> . A count of successes in <code>trials[state]</code> attempts,
	each with <code>p[state]</code> .
BinomialEmission.n_states	Hidden states this family emits from.
BinomialEmission.is_	True: the support is $\{0, \dots, n\}$.
discrete	
BinomialEmission.	Floating point, since <code>lgamma</code> consumes the counts.
observation_dtype	
BinomialEmission.trials	Per-state n , shape $(n_states,)$.
BinomialEmission.	Per-state p , shape $(n_states,)$.
probability	
BinomialEmission.mean	$n p$.

<code>BinomialEmission.</code>	$n p (1 - p)$, strictly below the mean.
<code>variance</code>	
<code>BinomialEmission.sample</code>	Draw one count per entry of <code>states</code> .
<code>BinomialEmission.log_</code>	$\log C(n, y) + y \log p + (n - y) \log(1 - p)$.
<code>density</code>	
<code>BinomialEmission.bregman_</code>	n times the Kullback-Leibler divergence of y / n from p .
<code>divergence</code>	
<code>BinomialEmission.</code>	Raise if an observation is not an integer in $[0, \max(\text{trials})]$.
<code>validate</code>	
<code>BinomialEmission.</code>	$p = \text{weighted mean} / n$, in closed form; n is not estimated.
<code>reestimate</code>	
<code>BinomialEmission.</code>	The per-state mean and variance, both in observation units.
<code>alignment_key</code>	
<code>BinomialEmission.named_</code>	probability. The trial count is a constant, not a fitted value.
<code>parameters</code>	
<code>BetaBinomialEmission</code>	<i>class</i> . A count of successes in <code>trials[state]</code> attempts with a random rate.
<code>BetaBinomialEmission.n_</code>	Hidden states this family emits from.
<code>states</code>	
<code>BetaBinomialEmission.is_</code>	True: the support is $\{0, \dots, n\}$.
<code>discrete</code>	
<code>BetaBinomialEmission.</code>	Floating point, since <code>lgamma</code> consumes the counts.
<code>observation_dtype</code>	
<code>BetaBinomialEmission.</code>	Per-state n , shape $(n_states,)$.
<code>trials</code>	
<code>BetaBinomialEmission.</code>	Per-state a , shape $(n_states,)$.
<code>alpha</code>	
<code>BetaBinomialEmission.</code>	Per-state b , shape $(n_states,)$.
<code>beta</code>	
<code>BetaBinomialEmission.</code>	$a + b$: how close this is to a binomial. Larger is closer.
<code>concentration</code>	
<code>BetaBinomialEmission.</code>	$n a / (a + b)$.
<code>mean</code>	
<code>BetaBinomialEmission.</code>	$n p (1 - p) (n + a + b) / (1 + a + b)$, above the binomial's.
<code>variance</code>	
<code>BetaBinomialEmission.</code>	Draw a rate from the Beta, then a binomial count at that rate.
<code>sample</code>	
<code>BetaBinomialEmission.log_</code>	$\log C(n, y) + \log B(y + a, n - y + b) - \log B(a, b)$.
<code>density</code>	
<code>BetaBinomialEmission.</code>	The log-density gap to the best rate at this concentration.
<code>bregman_divergence</code>	
<code>BetaBinomialEmission.</code>	Raise if an observation is not an integer in $[0, \max(\text{trials})]$.
<code>validate</code>	
<code>BetaBinomialEmission.</code>	Alternating bisection for (a, b) ; see <code>_solve_beta_binomial</code> .
<code>reestimate</code>	
<code>BetaBinomialEmission.</code>	The per-state mean and variance, both in observation units.
<code>alignment_key</code>	
<code>BetaBinomialEmission.</code>	α and β . The trial count is conditioned on, never fitted.
<code>named_parameters</code>	
<code>CountPairEmission</code>	<i>class</i> . A total count and the successes within it, as one observation.

<code>CountPairEmission.joint</code>	Whether the success channel's trials are the drawn total.
<code>CountPairEmission.n_</code>	Hidden states this family emits from.
<code>states</code>	
<code>CountPairEmission.is_</code>	True: the support is a pair of non-negative integers.
<code>discrete</code>	
<code>CountPairEmission.</code>	Floating point, as both channels' <code>lgamma</code> terms need.
<code>observation_dtype</code>	
<code>CountPairEmission.total</code>	The total channel, as the family it is.
<code>CountPairEmission.alpha</code>	The success channel's <code>a</code> , shape <code>(n_states,)</code> .
<code>CountPairEmission.beta</code>	The success channel's <code>b</code> , shape <code>(n_states,)</code> .
<code>CountPairEmission.</code>	<code>a + b</code> : how close the success channel is to a binomial.
<code>concentration</code>	
<code>CountPairEmission.rate</code>	<code>a / (a + b)</code> , the expected success fraction of a trial.
<code>CountPairEmission.trials</code>	The independent form's fixed trial count; None in the joint form.
<code>CountPairEmission.mean</code>	Per-state mean of each channel, shape <code>(n_states, 2)</code> .
<code>CountPairEmission.</code>	Per-state variance of each channel, shape <code>(n_states, 2)</code> .
<code>variance</code>	
<code>CountPairEmission.sample</code>	Draw one <code>(total, successes)</code> pair per entry of <code>states</code> .
<code>CountPairEmission.log_</code>	Score every pair under every state.
<code>density</code>	
<code>CountPairEmission.</code>	The two channels' divergences, summed as their log-densities are.
<code>bregman_divergence</code>	
<code>CountPairEmission.</code>	Raise if the pairs are not counts, or a success count is impossible.
<code>validate</code>	
<code>CountPairEmission.</code>	The M step: each channel's own, on the trials the form supplies.
<code>reestimate</code>	
<code>CountPairEmission.</code>	Both channels' mean and variance, shape <code>(n_states, 4)</code> .
<code>alignment_key</code>	
<code>CountPairEmission.named_</code>	<code>dispersion</code> , <code>mean</code> , <code>alpha</code> and <code>beta</code> .
<code>parameters</code>	
<code>identifiable_concentration_</code>	The concentration above which this much data cannot tell
<code>bound</code>	<code>a + b</code> from infinity.
<code>identifiable_dispersion_</code>	The dispersion above which this much data cannot tell <code>r</code>
<code>bound</code>	from infinity.
<code>pooled_variance_floor</code>	The variance below which a Gaussian state has collapsed onto a point.

2.5 enumeration

One enumeration, and one policy for when it is too large to attempt.

<code>refuse_oversized</code>	Raise if enumerating <code>count</code> things is past the limit.
<code>configurations</code>	Every assignment of <code>n_states</code> states to <code>n_sites</code> sites, in lexicographic order.
<code>posterior</code>	$(\log Z, p)$ over an enumeration: the shifted exponential, summed and normalized.
<code>site_marginals</code>	Per-site sums of <code>weights</code> over the state each configuration puts there.
<code>argmax</code>	The first index of the largest value: ties resolve to the lexicographically first configuration.

<code>enumerated_optimum</code>	The highest-scoring configuration and its score, by trying every one.
---------------------------------	---

2.6 fixtures

Reading a declared fixture, once, for every model that has one.

<code>Scale</code>	<i>class</i> . Which time budget a fixture is sized for.
<code>load_declared</code>	Read a fixture yaml and check that every required field is present.
<code>Params</code>	<i>class</i> . A fixture's declared truth, built from the mapping its yaml declares.
<code>Params.from_declared</code>	Validate <code>declared</code> field by field and build the truth.
<code>load_params</code>	Read a fixture yaml and build its declared truth as <code>kind</code> .

2.7 incidence

One compressed-row incidence, for every relation the package stores.

<code>SparseIncidence</code>	<i>class</i> . A relation between <code>n_rows</code> and <code>n_cols</code> indices, row-major.
<code>SparseIncidence.from_pairs</code>	Build from the entries (<code>rows[e]</code> , <code>columns[e]</code>) in any order.
<code>SparseIncidence.n_entries</code>	The entries of the relation.
<code>SparseIncidence.degrees</code>	Entries per row, length <code>n_rows</code> .
<code>SparseIncidence.rows</code>	The row owning each entry, length <code>n_entries</code> : <code>offsets</code> expanded.
<code>SparseIncidence.row</code>	The columns of row <code>index</code> , a view into <code>indices</code> .
<code>SparseIncidence.gather</code>	<code>values</code> , one per caller pair, read in this layout's entry order.
<code>SparseIncidence.transpose</code>	The relation the other way round, and the permutation that reads it.
<code>SparseIncidence.dense</code>	The relation as an (<code>n_rows</code> , <code>n_cols</code>) count array, for small cases.

2.8 inputs

What a recorded measurement is a function of: its code, and its libraries.

<code>module_closure</code>	The source files of <code>modules</code> and everything they import, transitively.
<code>library_versions</code>	<code>name==version</code> per library, <code>absent</code> for one not installed.

2.9 log

The run logger: elapsed time and the current phase on every line, and warnings that say it once (issue #311).

<code>RunLogger</code>	<i>class</i> . A logger whose phase is shared by every logger in the process, with once-only messages.
<code>RunLogger.runtime_phase</code>	The phase the run is in, or <code>None</code> between phases. Class state, by design.
<code>RunLogger.runtime_phase</code>	<i>No docstring</i> .

<code>RunLogger.warning_once</code>	<code>warning</code> the first time this message is seen on this logger, silence after.
<code>RunLogger.info_once</code>	<code>info</code> the first time this message is seen on this logger, silence after.
<code>PhaseFilter</code>	<i>class.</i> Stamp the shared phase onto every record, as (<code>phase</code>) or nothing.
<code>PhaseFilter.filter</code>	<i>No docstring.</i>
<code>RuntimeFormatter</code>	<i>class.</i> Format with the elapsed minutes since <code>start_time</code> , read from <code>clock</code> .
<code>RuntimeFormatter.format</code>	<i>No docstring.</i>
<code>get_logger</code>	The run logger for <code>name</code> , configured to write to <code>stream</code> and nowhere else.
<code>phase</code>	Set the shared runtime phase for the block, restoring the previous one on exit.

2.10 numerics

Numerical helpers with no model knowledge, importable from anywhere.

<code>sample_rows</code>	Draw one categorical index per entry of <code>rows</code> , from the row it selects.
<code>logsumexp</code>	<code>log(sum(exp(values)))</code> along <code>axis</code> , shifted by the maximum.
<code>constant_chain_kernel</code>	Whether a chain's transition kernel is the one-matrix form, refusing a third.

2.11 parallel

One seam for CPU parallelism over independent tasks (issue #344).

<code>map_tasks</code>	Apply <code>function</code> to every item, in parallel where asked, results in input order.
------------------------	---

2.12 ragged

Chains of unequal length, laid end to end with the lengths beside them.

<code>Ragged</code>	<i>class.</i> Segments of a flat array, addressed by their lengths.
<code>Ragged.offsets</code>	Where each segment starts, and where the last one ends.
<code>Ragged.n_segments</code>	How many chains the batch holds.
<code>Ragged.longest</code>	The longest segment, which is what a padded form is padded to.
<code>Ragged.rectangular</code>	Whether every segment is the same length.
<code>Ragged.segments</code>	Each segment as a view, in order; no copy is taken.
<code>Ragged.from_rectangular</code>	The ragged form of an <code>(n_segments, length, ...)</code> batch.
<code>Ragged.padded</code>	The batch as <code>(n_segments, longest, ...)</code> , and a live-position mask.

2.13 track

The run seam: Aim's own interface, a metrics set per problem, one record per sweep (issue #778).

<code>Run</code>	<i>class</i> . Where a run's numbers go: <code>Aim</code> 's three members, with <code>Aim</code> 's signatures.
<code>Run.track</code>	Append <code>value</code> to the sequence (<code>name</code> , <code>context</code>) at <code>step</code> .
<code>Run.close</code>	Close the run, flushing what it holds.
<code>NullRun</code>	<i>class</i> . The default: every method a no-op, so an untracked run pays one returned call a sweep.
<code>NullRun.track</code>	Discard <code>value</code> .
<code>NullRun.close</code>	Do nothing: there is nothing to flush.
<code>MemoryRun</code>	<i>class</i> . The referee for the tests: a <code>Run</code> that keeps what it is given, in order.
<code>MemoryRun.track</code>	Append (<code>step</code> , <code>value</code>) to the sequence (<code>name</code> , <code>context</code>).
<code>MemoryRun.close</code>	Do nothing: the record is the object.
<code>MemoryRun.series</code>	The sequence (<code>name</code> , <code>context</code>), as (<code>step</code> , <code>value</code>) in the order recorded.
<code>MemoryRun.last</code>	The last value of the sequence (<code>name</code> , <code>context</code>).
<code>as_aim</code>	<code>aim.Image(fig)</code> where <code>aim</code> imports, and <code>fig</code> itself where it does not.
<code>Metrics</code>	<i>class</i> . What a problem's state means, in numbers a person reads.
<code>Metrics.names</code>	The series this set records, in the order <code>__call__</code> returns them.
<code>TrackedOptimization</code>	<i>class</i> . The run and its metrics bound together: the one object a loop holds.
<code>TrackedOptimization.is_null</code>	Whether this records nothing, so a loop can skip assembling what it would pass.
<code>TrackedOptimization.record</code>	Record one iteration: the objective, the metrics of <code>state</code> , and the diagnostics.
<code>TrackedOptimization.record_cost</code>	Record what the run cost the machine: peak resident bytes, and the bytes its state holds.
<code>current</code>	The <code>TrackedOptimization</code> of the enclosing <code>track</code> block, or <code>NULL</code> .
<code>track</code>	Run the block recording into <code>run</code> , restoring the previous binding on exit.
<code>peak_rss_bytes</code>	The process's peak resident set size, in bytes.

3 learn/

3.1 learn/__init__

Reinforcement learning over discrete search problems.

No public names.

3.2 learn/actor_critic

Actor-critic: REINFORCE with the critic as its baseline (issue #313).

<code>ActorCriticTraining</code>	<i>class</i> . The outcome of an actor-critic run.
<code>advantage_surrogate_loss</code>	$-(1/N) \sum \text{episodes} \sum_t \log \pi(a_t s_t)$ <code>A.t</code> : minus the score-function estimator with the given advantages.
<code>critic_advantages</code>	$G_t - V(s_t)$ per step per episode, the critic read without gradient.

<code>actor_critic</code>	Alternate a critic refit and one advantage-weighted policy step, <code>iterations</code> times.
---------------------------	---

3.3 learn/arena

One table per problem: five learners on one axis, each stating what it spent.

<code>Scoring</code>	<i>class</i> . Which state in an episode decides whether it reached the optimum.
<code>Outcome</code>	<i>class</i> . A row against its problem's baseline: beat, match or lose.
<code>TrainingBudget</code>	<i>class</i> . What a learned row is given, in the three numbers its trainer takes.
<code>TrainingBudget.decisions</code>	Decisions the training run costs, at most.
<code>Streams</code>	<i>class</i> . The four generators one row consumes, each the caller's own.
<code>Row</code>	<i>class</i> . One learner's result on one problem, with what it cost.
<code>Row.fraction</code>	Reached over offered.
<code>Row.against</code>	Beat, match or lose against <code>baseline</code> .
<code>reached_optimum</code>	Whether <code>episode</code> reached <code>optimum</code> , under <code>scoring</code> .
<code>evaluations</code>	Actions <code>episode</code> scored: the sum of the neighbourhood at each decision.
<code>Learner</code>	<i>class</i> . One row of the table: how to train, and what to roll out afterwards.
<code>Learner.trains</code>	Whether this row has a training run to spend a budget on.
<code>Learner.episodes</code>	Train if this row trains, then the runs each of <code>starts</code> buys.
<code>row</code>	One learner's row: its success fraction and what it cost.
<code>table</code>	Every registered row on one problem, in <code>LEARNERS</code> order.

3.4 learn/canonical

Canonical control problems, with an exact optimum written a second way.

<code>SweepError</code>	<i>class</i> . Raised when value iteration did not reach its residual.
<code>ValueFunction</code>	<i>class</i> . <code>V*</code> over an enumerated state set, and what it cost to reach.
<code>ValueFunction.optimal_actions</code>	Every action attaining the maximum at <code>state</code> .
<code>reachable_states</code>	Every state reachable from <code>start</code> , breadth first and in that order.
<code>value_iteration</code>	<code>V*</code> by sweeping the Bellman optimality operator, undiscounted.
<code>shortest_path_length</code>	Decisions on the shortest route from <code>start</code> to a terminal state.
<code>ChainMdp</code>	<i>class</i> . <code>n</code> states in a line: a small reward at the near end, the prize at the far.
<code>ChainMdp.reset</code>	The near end, always: the fixture's point is the walk, not the start.
<code>ChainMdp.actions</code>	<i>No docstring.</i>
<code>ChainMdp.step</code>	<i>No docstring.</i>

<code>ChainMdp.features</code>	<code>(len(actions), 2)</code> : the step's reward, and how far it goes out.
<code>ChainMdp.n_features</code>	<i>No docstring.</i>
<code>ChainMdp.is_terminal</code>	<i>No docstring.</i>
<code>GridWorld</code>	<i>class.</i> A rectangle walked one cell at a time, with one goal and a step cost.
<code>GridWorld.reset</code>	A uniform non-goal cell, so credit assignment is measured from anywhere.
<code>GridWorld.actions</code>	<i>No docstring.</i>
<code>GridWorld.step</code>	<i>No docstring.</i>
<code>GridWorld.features</code>	<code>(len(actions), 2)</code> : the step's reward, and the distance it leaves.
<code>GridWorld.n_features</code>	<i>No docstring.</i>
<code>GridWorld.is_terminal</code>	<i>No docstring.</i>
<code>CliffWalk</code>	<i>class.</i> A grid whose bottom row, between the start and the goal, ends the episode.
<code>CliffWalk.start</code>	The bottom-left cell.
<code>CliffWalk.goal</code>	The bottom-right cell.
<code>CliffWalk.is_cliff</code>	Whether <code>state</code> is one of the bottom row's interior cells.
<code>CliffWalk.reset</code>	The bottom-left cell, always: the cliff is defined relative to it.
<code>CliffWalk.actions</code>	<i>No docstring.</i>
<code>CliffWalk.step</code>	<i>No docstring.</i>
<code>CliffWalk.features</code>	<code>(len(actions), 2)</code> : the step's reward, and the distance it leaves.
<code>CliffWalk.n_features</code>	<i>No docstring.</i>
<code>CliffWalk.is_terminal</code>	<i>No docstring.</i>
<code>TowersOfHanoi</code>	<i>class.</i> <code>d</code> disks over three pegs; the state is which peg each disk sits on.
<code>TowersOfHanoi.start</code>	Every disk on peg zero.
<code>TowersOfHanoi.goal</code>	Every disk on peg two.
<code>TowersOfHanoi.optimal_moves</code>	$2^d - 1$: the known shortest solution length.
<code>TowersOfHanoi.reset</code>	The classic start, every disk on one peg.
<code>TowersOfHanoi.actions</code>	<i>No docstring.</i>
<code>TowersOfHanoi.step</code>	<i>No docstring.</i>
<code>TowersOfHanoi.features</code>	<code>(len(actions), 2)</code> : the step's reward, and disks left off the goal peg.
<code>TowersOfHanoi.n_features</code>	<i>No docstring.</i>
<code>TowersOfHanoi.is_terminal</code>	<i>No docstring.</i>

3.5 learn/critic

The state-value critic, and the oracle that referees it (issue #313).

<code>state_features</code>	Mean and maximum of the available actions' features, and <code>log(1 + n_actions)</code> .
<code>n_state_features</code>	Width of <code>state_features</code> .
<code>Critic</code>	<i>class.</i> <code>V(s)</code> from state features: linear when <code>hidden</code> is <code>None</code> , a two-layer MLP otherwise.
<code>Critic.forward</code>	Values for a batch of state features, shape <code>(n,)</code> .

<code>CriticFit</code>	<i>class.</i> What fitting a critic recorded: the loss per step and the targets' count.
<code>monte_carlo_targets</code>	State features and returns-to-go for every non-final state in episodes.
<code>temporal_difference_targets</code>	State features and one-step bootstrapped targets $r_t + V(s_{t+1})$, zero past a terminal state.
<code>fit_critic</code>	Adam on the squared error over the whole batch, <code>steps</code> times.

3.6 learn/environment

The reinforcement-learning interface, and nothing that knows what it searches.

<code>Environment</code>	<i>class.</i> A discrete search problem an agent proposes moves in.
<code>Environment.reset</code>	Draw a starting state.
<code>Environment.actions</code>	The moves available in <code>state</code> .
<code>Environment.step</code>	Apply <code>action</code> and return the next state and its reward.
<code>Environment.features</code>	Features of each available action, shape $(\text{len}(\text{actions}), n)$.
<code>Environment.n_features</code>	Width of the feature vector <code>features</code> returns.
<code>Environment.is_terminal</code>	Whether the episode ends on reaching <code>state</code> .
<code>Episode</code>	<i>class.</i> One trajectory, recorded in full.
<code>Episode.total_reward</code>	The undiscounted return.
<code>Episode.returns_to_go</code>	$G_t = \sum_{u \geq t} R_u$ for each step, undiscounted.

3.7 learn/exact

Exact expected return by trajectory enumeration: the oracle for the agents.

<code>exact_expected_return</code>	$J(w)$: expected undiscounted return from <code>start</code> , enumerated exactly.
<code>exact_policy_gradient</code>	$\text{grad } J(w)$, by autodiff through the enumerated expectation.
<code>finite_difference_gradient</code>	Central differences of the enumerated J , entry by entry.
<code>exact_action_values</code>	$Q^\pi(s, a)$ for every available action: the reward plus V^π of the successor with one decision fewer.
<code>exact_optimal_value</code>	$V^*(s)$ over horizon decisions: finite-horizon value iteration by recursion.

3.8 learn/failure

Where a classical baseline first fails, which is where a gate can be argued.

<code>FailureProbe</code>	<i>class.</i> One reading: how often a method reached the target at one size.
<code>FailureProbe.fraction</code>	Reached over run. Zero starts is refused, so this never divides by zero.
<code>FailureProbe.truncated</code>	Whether the clock stopped the probe before the starts were spent.
<code>FailureCurve</code>	<i>class.</i> A sweep over sizes, and the first size at which the baseline failed.

<code>FailureCurve.first_failure</code>	The first probe under the threshold, or <code>None</code> if none is.
<code>probe_failure</code>	Run <code>method</code> from <code>starts</code> seeded starts and count what reached <code>target</code> .
<code>failure_curve</code>	Sweep sizes and report where the baseline first fell below <code>threshold</code> .

3.9 learn/gym

Gymnasium compliance as an adapter over `learn.Environment`, not a rewrite of it (issue #322).

<code>GymnasiumEnvironment</code>	<i>class</i> . A <code>learn.Environment</code> as a <code>gymnasium.Env</code> .
<code>GymnasiumEnvironment.state</code>	The current state, in the wrapped environment's own type.
<code>GymnasiumEnvironment.environment</code>	The wrapped environment.
<code>GymnasiumEnvironment.reset</code>	Start an episode from <code>environment.reset</code> drawn under <code>seed</code> .
<code>GymnasiumEnvironment.step</code>	Take the <code>action</code> -th move of the current neighbourhood.

3.10 learn/hmm

Single-position search over a hidden Markov state path at known parameters.

<code>HmmEnvironment</code>	<i>class</i> . Search over hidden state paths, one position at a time.
<code>HmmEnvironment.n_states</code>	Hidden states available at each position.
<code>HmmEnvironment.length</code>	Positions in the path.
<code>HmmEnvironment.energy</code>	Joint log-probability of the path and the observed sequence.
<code>HmmEnvironment.reset</code>	Draw a uniform random path.
<code>HmmEnvironment.actions</code>	Every single-position change to a different hidden state.
<code>HmmEnvironment.step</code>	Apply a revision and return the new path and its reward.
<code>HmmEnvironment.features</code>	<code>(len(actions), 2)</code> : the change in transition and in emission terms.
<code>HmmEnvironment.n_features</code>	Two: the transition change and the emission change.
<code>HmmEnvironment.is_terminal</code>	Whether no single-position change raises the joint log-probability.
<code>HmmEnvironment.greedy_weights</code>	The weight vector whose policy is greedy, up to temperature.
<code>enumerate_paths</code>	Every hidden path, in lexicographic order.
<code>optimum</code>	The maximum-a-posteriori path, by exhaustive enumeration.

3.11 learn/keyed

Randomness inside a move, without giving up a deterministic `step`.

<code>state_key</code>	A 64-bit key for one <code>(state, action)</code> pair under one environment seed.
<code>keyed_generator</code>	The generator a move is entitled to at this state and action.
<code>KeyedMove</code>	<i>class</i> . A Monte Carlo move a deterministic <code>step</code> can call.
<code>KeyedMove.n_nodes</code>	Sites the move expects, checked against the environment's.

<code>KeyedMove.n_states</code>	Labels the move expects, checked against the environment's.
<code>KeyedMove.parametric</code>	Whether an action of this move names a site and a target label.
<code>KeyedMove.propose</code>	The successor labelling, and what the move cost in site visits.

3.12 learn/planning

Planning with a learned prior and value: PUCT search and expert iteration (issue #313).

<code>SearchResult</code>	<i>class</i> . What one search from a root recorded.
<code>SearchResult.distribution</code>	The visit counts normalized: the search-improved policy at the root.
<code>SearchResult.best</code>	Index of the most visited move, ties to the higher mean value.
<code>critic_leaf_value</code>	A leaf evaluator reading the critic, zero at a terminal state or with no decisions left.
<code>puct_search</code>	<code>n_simulations</code> PUCT descents from <code>state</code> , <code>horizon</code> decisions deep.
<code>PlannedEpisode</code>	<i>class</i> . One episode played by the planner: states, chosen actions, rewards, and the root distributions.
<code>PlannedEpisode.total_reward</code>	<i>No docstring</i> .
<code>plan_episode</code>	Play from <code>start</code> choosing each move by a search, sampled from the visit distribution or its argmax.
<code>ExpertIterationTraining</code>	<i>class</i> . The outcome of expert iteration: mean planned return per iteration, the losses, and the evaluations spent.
<code>expert_iteration</code>	Alternate planned episodes with fits of the policy to the visit distributions and of the critic to the returns.

3.13 learn/policy

The policy: a softmax over scored actions, and the gauge that entails.

<code>Policy</code>	<i>class</i> . What <code>snakes_and_ladders.learn.rollout.rollout</code> needs of a policy.
<code>Policy.sample</code>	Choose an action index from the scored actions.
<code>TrainablePolicy</code>	<i>class</i> . What the actor-critic and PPO trainers need: differentiable log-probabilities and the tensors behind them.
<code>TrainablePolicy.dtype</code>	<i>No docstring</i> .
<code>TrainablePolicy.parameters</code>	<i>No docstring</i> .
<code>TrainablePolicy.log_probabilities</code>	<i>No docstring</i> .
<code>TrainablePolicy.greedy</code>	<i>No docstring</i> .
<code>LinearPolicy</code>	<i>class</i> . Scores each action linearly in its features, then softmaxes.
<code>LinearPolicy.weights</code>	The learned parameter vector, with <code>requires_grad</code> set.
<code>LinearPolicy.n_features</code>	Width of the feature vector this policy consumes.
<code>LinearPolicy.dtype</code>	Working precision.
<code>LinearPolicy.parameters</code>	The tensors an optimizer updates: the one weight vector.

<code>LinearPolicy.set_weights</code>	Replace the weights in place, keeping the same leaf tensor.
<code>LinearPolicy.log_probabilities</code>	Log $\pi(a \mid s)$ over the actions <code>features</code> describes.
<code>LinearPolicy.sample</code>	Draw an action index from $\pi(\cdot \mid s)$.
<code>LinearPolicy.greedy</code>	The most probable action, which is the zero-temperature limit.
<code>MLPPolicy</code>	<i>class</i> . Scores each action by a small multilayer perceptron on its features, then softmaxes (issue #313).
<code>MLPPolicy.dtype</code>	<i>No docstring.</i>
<code>MLPPolicy.n_features</code>	<i>No docstring.</i>
<code>MLPPolicy.parameters</code>	<i>No docstring.</i>
<code>MLPPolicy.log_probabilities</code>	Log $\pi(a \mid s)$ over the actions <code>features</code> describes, as <code>LinearPolicy.log_probabilities</code> .
<code>MLPPolicy.sample</code>	<i>No docstring.</i>
<code>MLPPolicy.greedy</code>	<i>No docstring.</i>
<code>EpsilonGreedyPolicy</code>	<i>class</i> . Takes the wrapped policy's best action, except a fraction of the time.
<code>EpsilonGreedyPolicy.epsilon</code>	The exploration probability this policy was built with.
<code>EpsilonGreedyPolicy.epsilon</code>	<i>No docstring.</i>
<code>EpsilonGreedyPolicy.log_probabilities</code>	Log of the mixture this policy samples from: $(1 - \text{eps})$ on the greedy action plus eps / n everywhere.
<code>EpsilonGreedyPolicy.sample</code>	Choose an action index: uniform with probability <code>epsilon</code> .

3.14 learn/potts

A Potts environment searched by single-site flips: reference environment.

<code>PottsEnvironment</code>	<i>class</i> . Single-flip search over a 1-D Potts chain at known parameters.
<code>PottsEnvironment.on_graph</code>	The same environment over an arbitrary graph rather than a chain.
<code>PottsEnvironment.from_params</code>	Build the environment a Potts fixture describes.
<code>PottsEnvironment.n_states</code>	States available at each site.
<code>PottsEnvironment.chain_length</code>	Sites in the chain.
<code>PottsEnvironment.coupling</code>	The true J .
<code>PottsEnvironment.field</code>	The true h , as a copy.
<code>PottsEnvironment.energy</code>	$E(s)$: the unnormalized log-density of one configuration.
<code>PottsEnvironment.reset</code>	Draw a uniform random configuration.
<code>PottsEnvironment.actions</code>	Every single-site flip to a different state.
<code>PottsEnvironment.step</code>	Apply a flip and return the new configuration and its reward.
<code>PottsEnvironment.features</code>	$(\text{len}(\text{actions}), 2)$: the change in agreement, and in field.
<code>PottsEnvironment.n_features</code>	Two: the agreement change and the field change.

<code>PottsEnvironment.is_terminal</code>	Whether <code>state</code> is a local maximum under single flips.
<code>PottsEnvironment.greedy_weights</code>	The weight vector whose policy is greedy, up to temperature.
<code>enumerate_configurations</code>	Every configuration of the chain, in lexicographic order.
<code>optimum</code>	The global maximum of the environment, by exhaustive enumeration.

3.15 learn/potts_nd

A Potts lattice in N dimensions with a per-site field, searched by Monte Carlo moves.

<code>FeatureColumn</code>	<i>class.</i> What a feature column reads, and the arms it can vary in.
<code>PottsAction</code>	<i>class.</i> One action: a move, where it applies, and how hot.
<code>PottsNDEnvironment</code>	<i>class.</i> Search over an N-D Potts lattice at a known per-site field.
<code>PottsNDEnvironment.n_states</code>	q.
<code>PottsNDEnvironment.n_nodes</code>	V.
<code>PottsNDEnvironment.coupling</code>	The true J.
<code>PottsNDEnvironment.field</code>	The true h, as a copy, so a trained-against parameter cannot move.
<code>PottsNDEnvironment.ladder</code>	The temperatures an action may name.
<code>PottsNDEnvironment.sweep_visits</code>	Site visits one sweep costs, as <code>search.ground_state</code> charges it.
<code>PottsNDEnvironment.score</code>	$J * \text{agreements} + \sum_i h_i[s_i]$, which the agent maximizes.
<code>PottsNDEnvironment.reset</code>	A uniform random labelling.
<code>PottsNDEnvironment.action_code</code>	A distinct integer per action, for <code>state key</code> .
<code>PottsNDEnvironment.actions</code>	The scored candidates at <code>state</code> : every rung, a subset of the sites.
<code>PottsNDEnvironment.visits</code>	What <code>action</code> costs in site visits, realized rather than nominal.
<code>PottsNDEnvironment.step</code>	Apply <code>action</code> and return the successor and the change in score.
<code>PottsNDEnvironment.features</code>	$(\text{len}(\text{actions}), \text{n_features}())$: one column per reading that varies.
<code>PottsNDEnvironment.bond_probability</code>	$1 - \exp(-J / T)$ at rung: one at $T = 0$, exactly.
<code>PottsNDEnvironment.n_features</code>	One per column this arm's move set and ladder let vary.
<code>PottsNDEnvironment.is_terminal</code>	Never: a sampler has no local-optimum stopping rule.
<code>PottsNDEnvironment.icm_action</code>	The action one iterated-conditional-modes sweep is: a sweep at $T = 0$.
<code>PottsNDEnvironment.steepest_action</code>	The best single flip anywhere at $T = 0$: steepest ascent, not ICM.

3.16 learn/ppo

Proximal policy optimization with generalized advantage estimation (issue #313).

<code>PPOTraining</code>	<i>class</i> . The outcome of a PPO run: the same diagnostics as the actor-critic, plus the clipped fraction.
<code>generalized_advantages</code>	eq:gae at $\gamma = 1$: $A_t = \sum_l \lambda^l \delta_{t+l}$, $\delta_t = r_t + V(s_{t+1}) - V(s_t)$.
<code>episode_advantages</code>	GAE per episode, the critic read without gradient.
<code>ppo_loss</code>	eq:ppo-clip: minus the mean over episodes of $\sum_t \min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t)$.
<code>ppo</code>	Collect a batch, refit the critic, then <code>epochs</code> clipped policy steps on that batch.

3.17 learn/ranking

Learned surrogates for the two searches: the data they are fitted to, and the seam that ranks by them (issue #308).

<code>maximized_target</code>	The maximized log-likelihood of a topology: a fit per call.
<code>fixed_length_target</code>	The log-likelihood of a topology with every branch at <code>length</code> : one pruning pass per call.
<code>tree_node_tokens</code>	Per-node tokens and the tree's edges, for the graph model: a node carries its branch's token, the root zeros.
<code>tree_examples</code>	One example per (alignment, topology), grouped by alignment, scored by <code>target</code> .
<code>plug_in_offset</code>	The plug-in bound in total, from its per-site feature and the site count.
<code>mean_field_offset</code>	The mean-field bound in total, from its per-node feature and the node count.
<code>ground_state_offset</code>	One decoupled energy bound per instance: what a ground-state target is predicted above.
<code>lattice_instances</code>	<code>n_groups * per_group</code> instances of the declared family, grouped by draw.
<code>enumerated_log_partition_target</code>	$\log Z$ by enumeration: exact, and affordable only where $q \ll N$ is.
<code>strip_log_partition_target</code>	$\log Z$ by the column transfer matrix: exact on an open 2-D strip past enumeration.
<code>ground_state_target</code>	The energy alpha-expansion reaches from the per-site data optimum: one discrete solve per call.
<code>lattice_examples</code>	One example per (graph, field), scored by <code>target</code> , grouped as given or one group each.
<code>shuffle_children</code>	The same tree with every node's children in a random order: a symmetry of the problem, not of its spelling.
<code>LearnedTreeSurrogate</code>	<i>class</i> . A fit, or a calibrated bound made from one, as the <code>Surrogate</code> a search ranks by.
<code>LearnedTreeSurrogate.kind</code>	<i>No docstring.</i>

3.18 learn/reinforce

REINFORCE with a baseline: the score-function estimator `docs/tex/textbook.tex` states.

Training	<i>class</i> . The outcome of a training run.
surrogate_loss	A scalar whose gradient is the REINFORCE estimator, negated to minimize.
reinforce	Train policy in place by REINFORCE.

3.19 learn/relaxed

Gumbel-softmax over discrete chains: gradients where the answer is known.

RelaxationMode	<i>class</i> . Which Gumbel-softmax estimator to use.
RelaxedObjective	<i>class</i> . A discrete chain score and its extension to the simplex.
RelaxedObjective.n_sites	Sites in the chain.
RelaxedObjective.n_states	States available at each site.
RelaxedObjective.relaxed	The score on the simplex, differentiable in probabilities .
RelaxedObjective.discrete	The score of one configuration, computed without the relaxation.
RelaxedPotts	<i>class</i> . The Potts chain's score, extended to the simplex.
RelaxedPotts.n_sites	<i>No docstring.</i>
RelaxedPotts.n_states	<i>No docstring.</i>
RelaxedPotts.relaxed	<i>No docstring.</i>
RelaxedPotts.discrete	<i>No docstring.</i>
RelaxedHmmPath	<i>class</i> . An HMM hidden path's joint log-probability, extended to the simplex.
RelaxedHmmPath.n_sites	<i>No docstring.</i>
RelaxedHmmPath.n_states	<i>No docstring.</i>
RelaxedHmmPath.relaxed	<i>No docstring.</i>
RelaxedHmmPath.discrete	<i>No docstring.</i>
RelaxedOptimum	<i>class</i> . The result of gradient ascent on the relaxed objective.
one_hot	A configuration as a (L, k) corner of the simplex.
gumbel_softmax	One Gumbel-softmax sample, per row.
enumerate_optimum	The best configuration, by trying every one.
exact_expected_score	$E_q[\text{score}(s)]$ by enumerating every configuration.
exact_expected_gradient	$d E_q[\text{score}] / d \text{ logits}$, by autodiff through the enumeration.
estimate_gradient	The Gumbel-softmax gradient estimate, averaged over n_samples draws.
optimize	Gradient ascent on the logits, then read the configuration off.

3.20 learn/rollout

Generating episodes: by a policy, and by the greedy baseline it must beat.

rollout	Run one episode under policy .
greedy_rollout	Run one episode taking the best-rewarded action at every step.
greedy_restarts	Hill climbing restarted until max_steps decisions are spent.

3.21 learn/surrogate

Learned surrogates: point predictors, and the calibrated bounds made from them (issue #308).

Examples	<i>class.</i> Structures as tensors: one row of features and one target each, with optional tokens.
Examples.subset	The examples where mask is true, in order.
Examples.residual	The targets less the offset: what a model is fitted to.
Examples.with_offset	The same examples, predicted relative to offset .
concatenate	Examples from several sources as one dataset, groups kept distinct by offsetting.
Split	<i>class.</i> Boolean masks over one Examples , disjoint and covering it.
split_by_group	Assign whole groups to train, validation and test, in group order, at the given fractions.
LinearSurrogate	<i>class.</i> Affine in the feature vector: the baseline every other model must beat.
LinearSurrogate.forward	<i>No docstring.</i>
MLPSurrogate	<i>class.</i> A deep MLP on the feature vector.
MLPSurrogate.forward	<i>No docstring.</i>
SetSurrogate	<i>class.</i> Deep Sets: encode each token, sum, decode, with the feature vector joined before decoding.
SetSurrogate.forward	<i>No docstring.</i>
AttentionSurrogate	<i>class.</i> One self-attention block over the tokens, mean-pooled: permutation-equivariant because it has no positions.
AttentionSurrogate.forward	<i>No docstring.</i>
GraphSurrogate	<i>class.</i> Message passing over the structure's own graph, neighbours summed, then pooled.
GraphSurrogate.forward	<i>No docstring.</i>
Standardizer	<i>class.</i> Per-feature mean and scale from the training split, applied everywhere else.
Standardizer.of	<i>No docstring.</i>
Standardizer.apply	<i>No docstring.</i>
Standardizer.undo	<i>No docstring.</i>
Fitted	<i>class.</i> A trained predictor with the standardization it was trained under.
Fitted.predict	Predictions in the target's own units, the offset restored, no gradient.
fit_surrogate	Adam on the squared error, stopped when the validation error has not improved for patience epochs.
r_squared	$1 - \text{SS_res} / \text{SS_tot}$: the fraction of the target's variance the prediction explains.
argmax_agreement	Fraction of groups whose largest target the prediction also ranks first.
CalibratedBound	<i>class.</i> A point predictor shifted by a residual quantile, claiming a bound at coverage .
CalibratedBound.predict	<i>No docstring.</i>
calibrate	Shift fitted into a bound holding on a coverage fraction of calibration .
augment	Each structure plus copies transformed spellings of it, for training a model that is not itself invariant.
Stage	<i>class.</i> One rung of a curriculum: what to train on and what to validate on at that size.

Curriculum	<i>class</i> . The fit after each stage, so a later size can be scored zero-shot and after transfer.
curriculum	Fit through the stages in order, each continuing from the previous fit's weights.

3.22 learn/tabular

Tabular Q-learning and SARSA, which differ in one expression.

ActionValues	<i>class</i> . A learned Q table, with what was spent reaching it.
ActionValues.value	<code>max_a Q(state, a)</code> , and 0.0 at a terminal or unseen state.
ActionValues.greedy_actions	Every action attaining the maximum at <code>state</code> .
ActionValues.greedy_episode	Run the greedy policy from <code>start</code> , breaking ties by insertion order.
ActionValues.behaviour_episode	Run the epsilon-greedy policy the table was learned under.
ActionValues.largest_deviation	$\max_s \max_a Q(s, a) - V^*(s) $ over states visited enough times.
q_learning	Off-policy control: bootstrap from the greedy successor action.
sarsa	On-policy control: bootstrap from the action the behaviour policy takes.

3.23 learn/tree

The phylogenetic environment: tree search as the MDP `sec:policy-gradient` of `docs/tex/textbook.tex` states.

RewardModel	<i>class</i> . Which log-likelihood the reward is a difference of.
FeatureSet	<i>class</i> . Which columns <code>TreeEnvironment.features</code> returns per move.
with_uniform_branch_lengths	<code>topology</code> with every branch set to <code>branch_length</code> .
exchanged_subtrees	The leaf sets a move detaches and attaches, from the two topologies' splits.
standardize	Each column centred and scaled to unit spread across the rows.
TreeEnvironment	<i>class</i> . Tree search as a Markov decision process.
TreeEnvironment.evaluations	Distinct topologies scored so far, which is what a budget counts.
TreeEnvironment.feature_set	Which columns <code>features</code> returns.
TreeEnvironment.score	Log-likelihood of <code>topology</code> under this environment's reward model.
TreeEnvironment.parsimony	Fitch parsimony score of <code>topology</code> , memoized as <code>score</code> is.
TreeEnvironment.split_support	Fraction of sites compatible with <code>split</code> , memoized per split.
TreeEnvironment.reset	Draw a starting topology uniformly over the leaf set's topologies.
TreeEnvironment.actions	The neighbourhood of <code>state</code> , deduplicated.

<code>TreeEnvironment.step</code>	Move to <code>action</code> and return it with the improvement it bought.
<code>TreeEnvironment.raw_features</code>	The <code>FEATURE_NAMES</code> columns of the full set, unstandardized.
<code>TreeEnvironment.features</code>	<code>(len(actions), n_features())</code> : what the policy scores each move by.
<code>TreeEnvironment.n_features</code>	Width of <code>features</code> : one, or the seven of the full set.
<code>TreeEnvironment.is_terminal</code>	Whether no move out of <code>state</code> improves the score.

4 likelihood/

4.1 likelihood/__init__

Felsenstein pruning and hardware dispatch. See `CLAUDE.md` in this directory.

No public names.

4.2 likelihood/algebraic

Decoders for the algebraic codes: Reed–Solomon by syndromes, Hamming by its syndrome table.

<code>DecodingFailure</code>	<i>class</i> . More errors than the code can correct, so no decode is returned.
<code>syndromes</code>	$S_j = r(\alpha^j)$ for j in $1..n-k$; all zero iff a codeword.
<code>berlekamp_massey</code>	The shortest linear recursion the syndromes satisfy, as its locator.
<code>chien_search</code>	The error positions, by trying every field element as a root.
<code>decode</code>	Correct up to t symbol errors, or refuse.
<code>syndrome</code>	$H \cdot x$ over $GF(2)$, for a word of the code's length.
<code>hamming_correct</code>	Correct one error by reading the syndrome as a bit index.

4.3 likelihood/belief_propagation

Sum-product belief propagation on a Potts MRF, and the Bethe free energy.

<code>ConvergenceError</code>	<i>class</i> . Raised when the messages did not settle within the iteration cap.
<code>BeliefPropagationResult</code>	<i>class</i> . Beliefs and the Bethe estimate, from a run that converged.
<code>belief_propagation</code>	Run sum-product to convergence, or refuse.

4.4 likelihood/blocks

The block-frequency bound: exact where blocks repeat, bounded where they do not (issue #408).

<code>Interval</code>	<i>class</i> . What the bound returns: two ends, and what it cost to get them.
<code>Interval.extrapolated</code>	The exact half scaled to the whole alignment: a point estimate, not a bound.

<code>Interval.width</code>	<code>upper - lower</code> : zero where nothing was bounded.
<code>Interval.contains</code>	Whether <code>value</code> lies inside, allowing <code>tolerance</code> relative slack.
<code>site_log_likelihood_extremes</code>	Bounds on $\log \Pr(\mathbf{x})$ holding for every column $\mathbf{x}$, in one pass over the tree.
<code>block_frequency_interval</code>	An interval containing the log-likelihood, exact on frequent blocks.
<code>BlockFrequencyBound</code>	<i>class</i> . The block-frequency interval as a <code>Surrogate</code> .
<code>BlockFrequencyBound.kind</code>	Which end this surrogate reports.
<code>BlockFrequencyBound.lengths</code>	The feasible branch lengths the interval is evaluated at.
<code>BlockFrequencyBound.interval</code>	The whole interval, and what it cost, at <code>lengths</code> .

4.5 likelihood/brute force

Direct marginalization: the independent oracle pruning is tested against.

<code>brute_force_log_likelihood</code>	Total log-likelihood by direct marginalization over internal states.
---	--

4.6 likelihood/convolutional

Decoding one convolutional code: BCJR, Viterbi, and the enumeration oracle.

<code>TrellisDecoding</code>	<i>class</i> . What one BCJR pass returns.
<code>bcjr</code>	Forward-backward over the trellis, with the observation on the edge.
<code>viterbi</code>	The single most likely input sequence: max-product on the same trellis.
<code>ExactTrellisDecoding</code>	<i>class</i> . The answers enumeration over all 2^{**K} messages gives.
<code>enumerate_messages</code>	Every K-bit message, as a $(2^{**K}, K)$ uint8 array.
<code>exact_bitwise_posterior</code>	Bit posteriors of a terminated convolutional code, by enumeration.

4.7 likelihood/convolutional_rust

Rust log-MAP BCJR (`snakes_and_ladders.oxi_snakes_and_ladders.bcjr_forward_backward`), pinned against `snakes_and_ladders.likelihood.convolutional.bcjr`, the NumPy oracle (`likelihood/CLAU` "The reference implementation is the oracle and it stays").

<code>bcjr</code>	One forward-backward pass over the trellis, computed in Rust.
-------------------	---

4.8 likelihood/css

Decoding a CSS code: the criterion degeneracy forces, and its exact oracle.

<code>decode_succeeds</code>	Whether correcting <code>error</code> with <code>estimate</code> leaves the codespace fixed.
<code>ErrorCosets</code>	<i>class</i> . Every error of a code, partitioned by syndrome and then by coset.

<code>error_cosets</code>	Enumerate every X error and group it by syndrome, then by coset.
<code>SyndromeDecoding</code>	<i>class</i> . One decode of a CSS code, and which of the two failures it is.
<code>decode_syndrome</code>	<code>sec:ldpc</code> 's decoder on a CSS code, scored by <code>decode_succeeds</code> .
<code>LogicalErrorRate</code>	<i>class</i> . What a decoder achieved over one seeded run, split by how it failed.
<code>LogicalErrorRate.failures</code>	Decodes that did not leave the codespace fixed.
<code>LogicalErrorRate.rate</code>	The share of trials that failed, either way.
<code>measure_logical_error_rate</code>	Draw trials errors and decode each, counting the two failures apart.

4.9 likelihood/device

Device selection and the cross-device agreement tolerance.

<code>select_device</code>	Choose a device from what is available, preferring the fastest.
<code>available_device</code>	The device this machine should run on.
<code>default_dtype</code>	The widest dtype device supports.
<code>cross_device_rtol</code>	The relative tolerance a comparison across dtypes is held to.

4.10 likelihood/distance

Pairwise distances from an alignment, with the variance a caller needs.

<code>DistanceKind</code>	<i>class</i> . Which estimator <code>distance_matrix</code> applies to every pair.
<code>Distance</code>	<i>class</i> . One pair's distance estimate and its delta-method variance.
<code>Distance.standard_error</code>	Square root of the variance.
<code>pair_counts</code>	The k x k joint count matrix of two aligned sequences.
<code>multinomial_delta_variance</code>	Delta-method variance of a statistic of multinomial proportions.
<code>jukes_cantor_distance</code>	The closed-form inversion of <code>eq:jc</code> from the fraction of differing sites, <code>eq:jc-distance</code> .
<code>log_det_distance</code>	The log-det distance of Steel (1994) from the pair frequency matrix, <code>eq:logdet</code> .
<code>distance_matrix</code>	Every pair's distance and variance, over the taxa in sorted order.
<code>tree_distances</code>	The additive distances a tree's branch lengths imply, leaf to leaf.

4.11 likelihood/features

Features a learned surrogate reads: cheap, model-aware, and invariant to how a structure is spelt (issue #308).

<code>tree_features</code>	One vector per topology, in <code>TREE_FEATURE_NAMES</code> order, per-site where it scales with sites.
<code>tree_tokens</code>	One row per branch, in <code>TREE_TOKEN_NAMES</code> order: its least-squares length, how evenly its split divides the taxa, and the mean distance across and within the split.
<code>lattice_features</code>	One vector per lattice, in <code>LATTICE_FEATURE_NAMES</code> order, per node where it scales with size.
<code>lattice_tokens</code>	One row per node, in <code>LATTICE_TOKEN_NAMES</code> order: its degree, the sum and absolute sum of its couplings, and three summaries of its own field.
<code>tree_adjacency</code>	Node names and the undirected edge list of the topology, by index into the names.

4.12 likelihood/forward_backward

Forward–backward as an evaluator, not as Baum–Welch’s internals (issue #173).

<code>ForwardBackward</code>	<i>class</i> . What the two passes return.
<code>step_kernels</code>	One transition kernel per step, and the matrix to use instead if there is one.
<code>forward_backward</code>	Run both passes on one chain.
<code>sample_path</code>	One draw of the hidden path from its posterior: forward filter, backward sample.

4.13 likelihood/hadamard

Hadamard conjugation: the exact spectral inversion for the two-state symmetric model.

<code>walsh_hadamard</code>	H v for the Sylvester–Hadamard matrix of order <code>len(v)</code> , in $O(N \log N)$.
<code>binary_recoding</code>	A k -state Jukes–Cantor alignment as a two-state symmetric one.
<code>recoding_scale</code>	The factor by which <code>binary_recoding</code> scales a branch length: $k / (2 (k - 1))$.
<code>sequence_spectrum</code>	The pattern frequencies of a two-state alignment, indexed by subset.
<code>hadamard_conjugation</code>	$q = H^{-1} \log(H s)$: the edge spectrum a sequence spectrum implies.
<code>expected_spectrum</code>	$s = H^{-1} \exp(H q)$: the forward direction, the exact spectrum of an edge spectrum.
<code>edge_spectrum</code>	A tree’s branch lengths as an edge spectrum, zero on every split it lacks.
<code>split_weights</code>	The edge spectrum as split weights, keyed as <code>leaf_bipartitions</code> keys splits.
<code>closest_tree</code>	The binary tree on the $n - 3$ largest mutually compatible split weights.

4.14 likelihood/hmm_paths

Every hidden path, enumerated: the oracle two decoders are separated by.

<code>PathEnumeration</code>	<i>class.</i> Both decodings of one observation sequence, and the evidence.
<code>PathEnumeration.decoders_agree</code>	Whether the two decodings coincide site for site.
<code>emission_log_density</code>	Score every observation under every hidden state.
<code>path_log_probability</code>	$\log P(\text{path}, \text{observations})$ from the definition, term by term.
<code>enumerate_hidden_paths</code>	Both decodings, by summing and maximizing over all $k \times T$ paths.

4.15 likelihood/ldpc

Decoding a parity-check code: sum-product, min-sum, and the enumeration oracle.

<code>DecodingAlgorithm</code>	<i>class.</i> The check update, and therefore which MAP the decision is.
<code>MapEstimate</code>	<i>class.</i> Which decoding a hard decision is.
<code>Decoding</code>	<i>class.</i> What the decoder returns.
<code>CodeMetrics</code>	<i>class.</i> What a decoder's hard decision means: how wrong it is, and whether it is a codeword (issue #778).
<code>decode</code>	Iterative decoding from channel log-likelihood ratios.
<code>ExactDecoding</code>	<i>class.</i> The answers enumeration gives, for the decoder to be held to.
<code>enumerate_codewords</code>	Every codeword, as a $(2 \times k, n_{\text{bits}})$ uint8 array.
<code>exact_decoding</code>	Bit posteriors and the maximum-likelihood codeword by enumeration.
<code>erasure_density_evolution</code>	<code>eq:density-evolution:</code> the erasure probability of a variable-to-check message on a cycle-free (<code>column_weight</code> , <code>row_weight</code>) code, per iteration.
<code>erasure_threshold</code>	The BP threshold <code>epsilon*</code> of the ensemble on the erasure channel.

4.16 likelihood/message_passing

Sum-product and max-product on a factor graph, exact on a tree.

<code>ConvergenceError</code>	<i>class.</i> Flooding did not settle within the iteration cap.
<code>Marginals</code>	<i>class.</i> What message passing returns.
<code>Marginals.exact</code>	Whether the numbers above are exact: the pre-#592 spelling, kept.
<code>sum_product</code>	Marginals and $\log Z$: exact under the tree schedule, Bethe under flooding.
<code>max_product</code>	The MAP assignment by max-marginals, and the max-marginals themselves.

4.17 likelihood/message_passing_reference

The dictionary-per-message sum-product: the oracle for `message_passing`.

<code>sum_product</code>	Marginals and $\log Z$: exact under the tree schedule, Bethe under flooding.
<code>max_product</code>	The MAP assignment by max-marginals, and the max-marginals themselves.

4.18 likelihood/message_passing_rust

Rust tree schedule (`snakes_and_ladders.oxi_snakes_and_ladders.tree_message_passing`), pinned against `snakes_and_ladders.likelihood.message_passing.sum_product` on its Python route, the NumPy oracle (`likelihood/CLAUDE.md`, "The reference implementation is the oracle and it stays"), which is itself pinned against `snakes_and_ladders.likelihood.message_passing_reference`.

<code>tree_messages</code>	The tree schedule's two passes, computed in Rust.
----------------------------	---

4.19 likelihood/mixture_assignments

Every component assignment, enumerated: the brute force a mixture's E step is held to.

<code>AssignmentEnumeration</code>	<i>class</i> . The mixture's evidence and E step, summed over every assignment.
<code>enumerate_mixture_assignments</code>	Sum the mixture's joint over all <code>k ** n</code> component assignments.

4.20 likelihood/objective

The phylogenetic instance of `snakes_and_ladders.opt.objective.Objective`.

<code>BranchLengthObjective</code>	<i>class</i> . Negative log-likelihood of an alignment as a function of branch lengths.
<code>BranchLengthObjective.topology</code>	The topology held fixed, as given; its branch lengths are not read.
<code>BranchLengthObjective.k</code>	Number of states.
<code>BranchLengthObjective.alignment</code>	The observed states per taxon, for an initializer that reads the data.
<code>BranchLengthObjective.parameter_names</code>	Names of the *estimable* parameters, in <code>theta</code> order.
<code>BranchLengthObjective.n_parameters</code>	Number of estimable parameters.
<code>BranchLengthObjective.initial</code>	A short, uninformative branch length everywhere.
<code>BranchLengthObjective.constrain</code>	The estimable branch lengths <code>theta</code> encodes, in <code>parameter_names</code> order.
<code>BranchLengthObjective.branch_lengths</code>	Expand <code>theta</code> to one length per branch, in <code>branch_order</code> .
<code>BranchLengthObjective.gradient</code>	Which route this objective's derivative comes from.
<code>BranchLengthObjective.fitted_tree</code>	The topology with <code>theta</code> 's branch lengths attached.
<code>BranchLengthObjective.theta_from</code>	The unconstrained vector whose <code>constrain</code> is named.
<code>BranchLengthObjective.theta_from_truth</code>	Place a tree's own branch lengths in the unconstrained coordinates.
<code>SubstitutionModelObjective</code>	<i>class</i> . Fits branch lengths, GTR exchangeabilities and <code>pi</code> together.
<code>SubstitutionModelObjective.topology</code>	The topology held fixed, as given; its branch lengths are not read.
<code>SubstitutionModelObjective.k</code>	Number of states.

`SubstitutionModelObjective`. The observed states per taxon, for an initializer that reads `alignment` the data.
`SubstitutionModelObjective`. Branch lengths, free exchangeabilities, and free `pi` entries.
`n_parameters`
`SubstitutionModelObjective`. Names in `theta` order, branch lengths first.
`parameter_names`
`SubstitutionModelObjective`. The normalized GTR rate matrix `theta` encodes.
`rate_matrix`
`SubstitutionModelObjective`. Uninformative: short branches, equal exchangeabilities,
`initial` uniform `pi`.
`SubstitutionModelObjective`. Branch lengths, the full exchangeability vector, and `pi`.
`constrain`
`SubstitutionModelObjective`. The unconstrained vector whose `constrain` is named.
`theta_from`
`SubstitutionModelObjective`. Place a known truth in the unconstrained coordinates.
`theta_from_truth`
`TreeMetrics` *class*. What a topology means: its likelihood, its parsimony score, its distance to truth (issue #778).

4.21 likelihood/parsimony

Parsimony: the criterion that is provably wrong in a known place.

`unit_step_matrix` The step matrix under which `sankoff_score` is `fitch_score`.
`fitch_score` Fewest state changes on `tau` explaining `alignment`, summed over sites (`eq:fitch`).
`sankoff_score` Minimum weighted change on `tau` explaining `alignment`, summed over sites (`eq:sankoff`).
`brute_force_parsimony_score` The same score, by enumerating every internal-node labelling.

4.22 likelihood/patterns

Site-pattern compression: the alignment as unique columns with integer weights (issue #408).

`SitePatterns` *class*. An alignment's distinct columns and how often each occurs.
`SitePatterns.n_patterns` Distinct columns.
`SitePatterns.compression_ratio` `n_sites / n_patterns`: the factor by which the column work falls.
`SitePatterns.alignment` The pattern table in the mapping from a backend's `alignment` takes.
`compress` Collapse identical alignment columns to distinct patterns with counts.
`compress_columns` Collapse identical columns of a raw block, adding the weights they carry.
`check_weights` Validate a backend's `weights` argument against its column count.

4.23 likelihood/polar

Successive cancellation on a polar code, and the list that fixes it.

<code>PolarDecoding</code>	<i>class</i> . What a polar decoder returns.
<code>decode_scl</code>	Successive-cancellation list decoding; <code>list_size = 1</code> is plain SC.
<code>crc_encode</code>	The <code>k</code> message bits a CRC-aided code carries: the payload, then its remainder.
<code>crc_checks</code>	Whether <code>message</code> 's last <code>r</code> bits are the remainder of its first <code>k - r</code> .
<code>decode_sc</code>	Plain successive cancellation: <code>decode_scl</code> at <code>list_size = 1</code> .
<code>transmitted</code>	The codeword a message produces, for a test that needs both ends.
<code>is_codeword</code>	Whether <code>bits</code> lies in the code: its frozen source positions are zero.

4.24 likelihood/potts

Exact evaluation of a Potts MRF: enumeration, and the strip transfer matrix.

<code>ExactPotts</code>	<i>class</i> . What enumeration establishes about a Potts MRF.
<code>log_weights</code>	Unnormalized log weight of each configuration.
<code>enumerate_potts</code>	Sum over every configuration: the oracle, exponential and deliberately so.
<code>strip_log_partition</code>	Exact $\log Z$ for an $N \times M$ strip, by transferring a whole column.

4.25 likelihood/pruning

Vectorized NumPy Felsenstein pruning – the oracle every backend is pinned against.

<code>log_likelihood</code>	Total log-likelihood of an alignment under the k -state Jukes-Cantor model.
-----------------------------	---

4.26 likelihood/pruning_analytic

Analytic gradient of Felsenstein pruning, behind one `torch.autograd.Function`.

<code>log_likelihood</code>	Total log-likelihood, differentiable w.r.t. <code>branch_lengths</code> .
-----------------------------	---

4.27 likelihood/pruning_common

The pruning routes' shared plumbing: one post-order, one leaf indicator, one rescale, one set of checks (issue #858).

<code>postorder</code>	Every node in the tree rooted at <code>root</code> , children before parents.
<code>check_pi_shape</code>	Refuse a root distribution that is not $(k,)$.
<code>check_alignment_covers</code>	Refuse an alignment missing a leaf of the topology.
<code>check_branch_lengths_shape</code>	Refuse a branch-length tensor that does not match <code>branch_order(tau)</code> .
<code>require_branch_length</code>	<code>node</code> 's branch length, refusing a non-root node that carries none.
<code>leaf_indicator_array</code>	The one-hot partial of a leaf's observed states, as a NumPy array.

<code>leaf_indicator</code>	The one-hot partial of a leaf’s observed states, as a tensor.
<code>rescale_partial</code>	Divide a partial by its per-site maximum, accumulating the logarithm.

4.28 likelihood/pruning_rust

Rust CPU Felsenstein pruning (`snakes_and_ladders.oxi_snakes_and_ladders.pruning_log_likelihood`) – pinned against `snakes_and_ladders.likelihood.pruning`, the NumPy oracle (`likelihood/CLAUDE.md`, "The NumPy reference is the oracle and it stays").

<code>log_likelihood</code>	Total log-likelihood of an alignment under the k-state Jukes-Cantor model.
-----------------------------	--

4.29 likelihood/pruning_torch

Differentiable PyTorch Felsenstein pruning – pinned against `snakes_and_ladders.likelihood.pruning`, the NumPy oracle (`CLAUDE.md`, "The NumPy reference is the oracle and it stays").

<code>branch_order</code>	Child-node names in <code>edges(tau)</code> order.
<code>branch_lengths_from_tree</code>	Read <code>tau</code> ’s own branch lengths into a tensor, ordered by <code>branch_order</code> .
<code>transition_probabilities</code>	$P(t)$ for every branch length in <code>t</code> , shape $(*t.shape, k, k)$.
<code>log_likelihood</code>	Total log-likelihood, differentiable w.r.t. <code>branch_lengths</code> .
<code>PartialCache</code>	<i>class</i> . Partial likelihoods of subtrees, keyed by what determines them.
<code>PartialCache.get</code>	<i>No docstring.</i>
<code>PartialCache.put</code>	<i>No docstring.</i>
<code>log_likelihood_cached</code>	<code>log_likelihood</code> without gradients, reusing cached subtree partials.

4.30 likelihood/ragged_rust

The ragged forward-backward in Rust, pinned to the NumPy oracle.

<code>posteriors</code>	Marginals, transition counts and per-segment evidence, in Rust.
<code>posteriors_oracle</code>	The same, one segment at a time through <code>forward_backward</code> .

4.31 likelihood/schedule

Which messages are sent, in what order, and when to stop.

<code>Guarantee</code>	<i>class</i> . What a schedule’s answer is worth, which is not always a yes or a no.
<code>MessageScheduleName</code>	<i>class</i> . The schedules by name, for a caller that does not want to import one.
<code>VariableSends</code>	<i>class</i> . Variable-to-factor messages of one (degree, cardinality), as edge rows.
<code>VariableBeliefs</code>	<i>class</i> . Variables of one (degree, cardinality) with all their edges, for the beliefs.

FactorSends	<i>class.</i> Factor-to-variable messages of one table shape along one axis.
Layout	<i>class.</i> The graph as contiguous edge arrays, and the groups a sweep runs over.
Layout.variable_sends	Group (variable, target edge) sends by (degree, cardinality).
Layout.factor_sends	Group (factor, axis) sends by (table shape, axis); None keeps every axis.
Layout.flooding_step	Every variable-to-factor send, then every factor-to-variable send.
Layout.root	The variable the tree passes are rooted at: the first, as the reference is.
Layout.tree_passes	The leaf-to-root sends grouped by height, and root-to-leaf by depth.
Layout.factor_neighbours	The factors sharing a variable with each factor, excluding itself.
Layout.sequential_steps	One step per factor: its incoming messages, then its outgoing ones.
Layout.variable_beliefs	<i>No docstring.</i>
MessageSchedule	<i>class.</i> An order over the messages, and what that order guarantees.
MessageSchedule.guarantee	What the answer is worth.
MessageSchedule.bounded	Whether the plan is finite, as against iterating to a tolerance.
MessageSchedule.requires_tree	Whether the schedule is refused on a graph with a cycle.
MessageSchedule.compiled	Whether <code>snakes_and_ladders.likelihood.message_passing_rust</code> runs this order.
MessageSchedule.damped	Whether a send is mixed with the value it replaces.
MessageSchedule.adaptive	Whether <code>steps</code> chooses its next send from the last send's residual.
MessageSchedule.sweep_length	How many steps make one sweep, so a residual is compared over a full pass.
MessageSchedule.steps	The sends, in order. May be infinite when <code>bounded</code> is false.
MessageSchedule.log_partition	$\log Z$, from whichever of the two routes this schedule has.
MessageSchedule.defined	The variables whose marginal this schedule leaves defined.
TreeMessageSchedule	<i>class.</i> Leaves to root, then root to leaves. Exact, and refused off a tree.
TreeMessageSchedule.guarantee	<i>No docstring.</i>
TreeMessageSchedule.requires_tree	<i>No docstring.</i>
TreeMessageSchedule.compiled	<i>No docstring.</i>
TreeMessageSchedule.steps	<i>No docstring.</i>
UpwardMessageSchedule	<i>class.</i> The leaf-to-root pass alone: Felsenstein pruning.
UpwardMessageSchedule.guarantee	<i>No docstring.</i>

UpwardMessageSchedule.	<i>No docstring.</i>
requires_tree	
UpwardMessageSchedule.	<i>No docstring.</i>
steps	
UpwardMessageSchedule.	scale plus the root's own normalizer: exactly what pruning
log_partition	returns.
UpwardMessageSchedule.	<i>No docstring.</i>
defined	
DownwardMessageSchedule	<i>class.</i> The root-to-leaf pass alone, from the priors.
DownwardMessageSchedule.	<i>No docstring.</i>
guarantee	
DownwardMessageSchedule.	<i>No docstring.</i>
requires_tree	
DownwardMessageSchedule.	<i>No docstring.</i>
steps	
DownwardMessageSchedule.	Not available, and reported as <code>nan</code> rather than guessed.
log_partition	
DownwardMessageSchedule.	<i>No docstring.</i>
defined	
FloodingMessageSchedule	<i>class.</i> Every message at once from the previous sweep's
	values: Jacobi.
FloodingMessageSchedule.	<i>No docstring.</i>
guarantee	
FloodingMessageSchedule.	<i>No docstring.</i>
bounded	
FloodingMessageSchedule.	<i>No docstring.</i>
sweep_length	
FloodingMessageSchedule.	<i>No docstring.</i>
steps	
SequentialMessageSchedule	<i>class.</i> One message at a time, each from the newest values:
	Gauss-Seidel.
SequentialMessageSchedule.	<i>No docstring.</i>
guarantee	
SequentialMessageSchedule.	<i>No docstring.</i>
bounded	
SequentialMessageSchedule.	<i>No docstring.</i>
steps	
ResidualMessageSchedule	<i>class.</i> The factor whose inputs moved most sends next:
	residual belief propagation.
ResidualMessageSchedule.	<i>No docstring.</i>
guarantee	
ResidualMessageSchedule.	<i>No docstring.</i>
bounded	
ResidualMessageSchedule.	<i>No docstring.</i>
adaptive	
ResidualMessageSchedule.	<i>No docstring.</i>
steps	
resolve	A <code>Schedule</code> from itself, its enum member, or its name.

4.32 likelihood/spatio_sequential

Exact quantities for the coupled spatio-sequential model, by enumeration.

<code>ExactSpatioSequential</code>	<i>class</i> . What enumeration returns.
<code>log_prior</code>	Unnormalized $\log p(l)$, the Potts term of <code>eq:joint</code> , per labelling, shape <code>(n_labellings,)</code> .
<code>log_chain</code>	$\log p(k_{\{.,m\}})$ per path, written out term by term, shape <code>(n_paths,)</code> .
<code>log_joint_at</code>	<code>eq:joint</code> at one assignment, up to $\log Z_{\text{Potts}}$, written out term by term.
<code>enumerate_spatio_sequential</code>	Sum <code>eq:joint</code> over every assignment.
<code>conditional_state_posterior</code>	$Q(k_{\{s,m\}} \mid l, x)$ by enumeration over paths, shape <code>(M, S, K)</code> .
<code>log_evidence_by_forward</code>	$\log p(x)$ by a route that shares no code with the enumeration.
<code>covariate_block</code>	<code>params.covariate</code> for one block of vertices, ready for a family.
<code>class_log_density</code>	Per class, the summed emission scores of its members, shape <code>(M, S, K)</code> .
<code>ClassPosteriors</code>	<i>class</i> . The E step of the coupled model, given a labelling.
<code>class_posteriors</code>	Forward-backward on every class's chain over its members' summed scores.
<code>external_field</code>	H _{nm} of the external-field equation of the textbook: minus the posterior-expected emission score, shape <code>(n_nodes, M)</code> .
<code>labelled_log_likelihood</code>	$\log p(x, l \mid \theta)$ with the chains marginalized, up to $\log Z_{\text{Potts}}$.
<code>map_labelling</code>	The labelling of highest $p(l \mid x)$, by enumeration: the oracle a label step is held to.
<code>marginal_log_likelihood_torch</code>	$\log p(x \mid l, \theta)$ as a differentiable scalar, through the forward recursion.

4.33 likelihood/spatio_sequential_rust

The coupled E step and its field in Rust, pinned to the NumPy oracle (issue #399).

<code>emission_tables</code>	Both channels' log-density for every class, state and table row.
<code>emission_rows</code>	The rows and the tables together, since neither is meaningful alone.
<code>class_posteriors</code>	Forward-backward on every class's chain over its members' summed scores.
<code>external_field</code>	$H[n, m]$, minus the posterior-expected emission score, shape <code>(n_nodes, M)</code> .
<code>labelled_log_likelihood</code>	$\log p(x, l \mid \theta)$ with the chains marginalized, up to $\log Z_{\text{Potts}}$.

4.34 likelihood/surrogate

Surrogates for an exact evaluation: bounds with proofs, and the seam a learned one fills (issue #308).

<code>jc_distances</code>	Pairwise Jukes–Cantor distances, $-(k-1)/k \log(1 - kp/(k-1))$, saturating where undefined.
<code>least_squares_lengths</code>	Non-negative branch lengths whose path lengths best fit distances, in <code>branch_order</code> .
<code>least_squares_residual</code>	$\sum (\text{path length} - \text{distance})^2$ at the given lengths: how tree-like the distances are.
<code>PlugInLikelihood</code>	<i>class</i> . Lower bound on the maximized log-likelihood: one pruning evaluation at least-squares lengths.
<code>PlugInLikelihood.lengths</code>	<i>No docstring</i> .
<code>prune_with_matrices</code>	Pruning with an arbitrary matrix per branch, summed over sites in the log domain.
<code>site_fitch_scores</code>	Fitch’s minimum change count per site, the recursion of <code>fitch_score</code> kept per column.
<code>ParsimonyUpperBound</code>	<i>class</i> . Upper bound on the Jukes–Cantor log-likelihood at every branch length, hence on its maximum.
<code>site_rows</code>	<i>field</i> as one row per site, widened in <code>torch</code> so the gradient survives.
<code>mean_field_log_partition</code>	Naive mean field: $\log Z \geq E_q[-E] + H(q)$ for the product q at its fixed point.
<code>tree_log_partition</code>	Exact $\log Z$ of a Potts model on a tree by one leaf-to-root pass, in the log domain.
<code>spanning_tree_log_partition</code>	Jensen’s upper bound on $\log Z$ over a distribution on spanning trees.
<code>decoupled_ground_energy</code>	$-\sum_n \max_m h[n, m] - \sum_e \max(J_e, 0)$: no configuration’s energy lies below it.
<code>decoupled_log_partition</code>	$\sum_n \log \sum \exp_m h[n, m]$: $\log Z$ with every coupling dropped.
<code>saturated_log_partition</code>	$N \log q - E_{\text{decoupled}}$: an upper bound on $\log Z$ at any size.
<code>ground_state_energy_bounds</code>	$[-U/\beta, (N \log q - L)/\beta]$ brackets the ground-state energy, from $L \leq \log Z(\beta) \leq U$.
<code>MeanFieldLogPartition</code>	<i>class</i> . The mean-field bound as a <code>Surrogate</code> over <code>(graph, field)</code> .
<code>SpanningTreeLogPartition</code>	<i>class</i> . The spanning-tree bound as a <code>Surrogate</code> over <code>(graph, field)</code> .

4.35 likelihood/turbo

Turbo decoding: two BCJR passes exchanging extrinsic log-likelihood ratios.

<code>TurboStreams</code>	<i>class</i> . One received word split into the streams each constituent decoder reads.
<code>split_streams</code>	Cut a received word’s ratios into the four streams of <code>eq:turbo-word</code> .
<code>decode_turbo</code>	Run <code>iterations</code> extrinsic exchanges and return the message decision.
<code>decode_turbo_per_iteration</code>	The message decision after each of <code>1 .. iterations</code> iterations.
<code>ExactTurboDecoding</code>	<i>class</i> . What enumerating all 2^{**K} messages says about a received word.
<code>exact_turbo_posterior</code>	Bit posteriors of the turbo code by enumeration over its messages.

<code>uncoded_bit_error_rate</code>	$Q(\sqrt{2 E_b / N_0})$: uncoded antipodal signalling, the analytic pin.
<code>noise_scale</code>	The Gaussian channel's <code>sigma</code> at an E_b / N_0 and a code rate.
<code>ErrorRates</code>	<i>class</i> . A waterfall point: what was measured, and how many frames it rests on.
<code>ErrorRates.bit_error_rate</code>	Errors over bits, per iteration count.
<code>ErrorRates.frame_error_rate</code>	Frames in error over frames, per iteration count.
<code>ErrorRates.bit_error_interval</code>	The one-sigma binomial interval half-width on <code>bit_error_rate</code> .
<code>measure_error_rates</code>	Simulate <code>frames</code> transmissions at one E_b / N_0 and count errors.

5 opt/

5.1 opt/__init__

Continuous parameter fitting by autodiff. See CLAUDE.md in this directory.

No public names.

5.2 opt/budget

One budgeted comparison, so every equal-budget claim is the same comparison.

<code>OverspendError</code>	<i>class</i> . A method reported spending more than its budget.
<code>Budget</code>	<i>class</i> . A cost ceiling in one declared unit.
<code>Outcome</code>	<i>class</i> . What one run of a method returns.
<code>restarts</code>	The random-restart baseline at the same budget.
<code>Restarts</code>	<i>class</i> . What <code>restarts</code> returns: a method, and picklable where <code>single</code> is.
<code>Comparison</code>	<i>class</i> . Every method on every instance, at one budget.
<code>Comparison.reached</code>	Per method, which instances it reached the reference on, shape <code>(n_instances,)</code> .
<code>Comparison.hits</code>	Instances on which each method reached the reference, within <code>tolerance</code> .
<code>Comparison.paired_p</code>	<code>mcnemar</code> on <code>first</code> 's and <code>second</code> 's hits, instance by instance.
<code>Comparison.mean_gap</code>	Mean of <code>best - reference</code> per method, zero when every instance is a hit.
<code>Comparison.table</code>	The markdown row set <code>STATUS.md</code> cites: hits, mean gap and spend per method.
<code>compare</code>	Run every method on every instance over the seeds, at one budget.
<code>mcnemar</code>	Exact two-sided McNemar p-value for two methods' hits on the same instances.

5.3 opt/constrain

Constraint maps: unconstrained reals in, feasible parameters out.

<code>log_simplex</code>	Map $n - 1$ unconstrained reals to n log-probabilities.
<code>free_from_log_simplex</code>	Invert <code>log_simplex</code> .

<code>positive</code>	Map unconstrained reals to strictly positive values.
<code>free_from_positive</code>	Invert <code>positive</code> .
<code>probability</code>	Map unconstrained reals to strictly interior probabilities.
<code>free_from_probability</code>	Invert <code>probability</code> .

5.4 `opt/em`

The alternation every expectation-maximization run here performs, once.

<code>em_loop</code>	Alternate <code>step</code> until the log-likelihood settles or the budget runs out.
----------------------	--

5.5 `opt/emission_mixture`

A mixture of count emissions: the Gaussian mixture with its family swapped.

<code>EmissionMixtureFit</code>	<i>class</i> . What one expectation-maximization run produced.
<code>expectation_maximization</code>	Fit a mixture of count emissions by EM.
<code>enumerated_posterior</code>	$P(\text{component} \mid \text{observations})$ summed over every joint labelling.
<code>CountPairSeeding</code>	<i>class</i> . Places a <code>CountPairEmission</code> component on an observed pair.
<code>plus_plus_start</code>	Seed the components by <code>Emission_Mixture++</code> (issue #306, <code>eq:kmeanspp</code>).
<code>uniform_start</code>	Seed the components on observations drawn uniformly, without replacement.

5.6 `opt/fit`

Gradient-based fitting, and the intervals that make a fit falsifiable.

<code>FitResult</code>	<i>class</i> . Outcome of a fit.
<code>fit</code>	Minimize <code>objective</code> by L-BFGS with a strong-Wolfe line search.
<code>observed_information</code>	Hessian of the objective at <code>theta</code> .
<code>parameter_covariance</code>	Inverse observed information: the asymptotic covariance of <code>theta</code> .
<code>constrained_standard_errors</code>	Delta-method standard errors of the <i>*constrained*</i> parameters.
<code>standard_errors_at</code>	Delta-method standard errors at a fit stated in the model's parameters.
<code>covers</code>	Whether a 95% Wald interval around <code>estimate</code> contains <code>truth</code> .
<code>MultiStartResult</code>	<i>class</i> . Every fit an initializer's starts produced, best first.
<code>fit_from</code>	Fit from every start an initializer offers, and report all of them.

5.7 `opt/hmm`

A discrete HMM: the second reference instance of `Objective`.

<code>HmmObjective</code>	<i>class</i> . Negative log-likelihood of observed sequences, by the forward algorithm.
---------------------------	---

<code>HmmObjective.emissions</code>	The categorical family <code>theta</code> 's emission block encodes.
<code>HmmObjective.initial</code>	A start that is uninformative but **not** symmetric.
<code>HmmObjective.constrain</code>	Split <code>theta</code> into log initial, transition and emission matrices.
<code>HmmObjective.theta_from_truth</code>	Place a known truth in the unconstrained coordinates.
<code>GaussianHmmObjective</code>	<i>class.</i> Negative log-likelihood of real-valued sequences from a Gaussian HMM.
<code>GaussianHmmObjective.variance_floor</code>	The floor the EM oracle refuses at, derived from these observations.
<code>GaussianHmmObjective.emissions</code>	The Gaussian family <code>theta</code> 's emission block encodes.
<code>GaussianHmmObjective.initial</code>	A start that is uninformative but **not** symmetric.
<code>GaussianHmmObjective.constrain</code>	Split <code>theta</code> into log transitions and the per-state mean and scale.
<code>GaussianHmmObjective.theta_from_truth</code>	Place a known truth in the unconstrained coordinates.
<code>PoissonHmmObjective</code>	<i>class.</i> Negative log-likelihood of count sequences from a Poisson-emission HMM.
<code>PoissonHmmObjective.emissions</code>	The Poisson family <code>theta</code> 's emission block encodes.
<code>PoissonHmmObjective.initial</code>	Uniform transitions; rates at quantiles of the pooled counts.
<code>PoissonHmmObjective.theta_from_truth</code>	Place a known (<code>pi</code> , <code>A</code> , <code>lambda</code>) in the unconstrained coordinates.
<code>BinomialHmmObjective</code>	<i>class.</i> Negative log-likelihood of bounded counts from a binomial-emission HMM.
<code>BinomialHmmObjective.emissions</code>	The binomial family <code>theta</code> 's emission block encodes.
<code>BinomialHmmObjective.initial</code>	Uniform transitions; success rates from quantiles of the counts.
<code>BinomialHmmObjective.theta_from_truth</code>	Place a known (<code>pi</code> , <code>A</code> , <code>p</code>) in the unconstrained coordinates.
<code>BetaBinomialHmmObjective</code>	<i>class.</i> Negative log-likelihood of bounded counts from a beta-binomial HMM.
<code>BetaBinomialHmmObjective.emissions</code>	The beta-binomial family <code>theta</code> 's emission block encodes.
<code>BetaBinomialHmmObjective.initial</code>	Uniform transitions; (<code>a</code> , <code>b</code>) from the quantile rates at unit concentration.
<code>BetaBinomialHmmObjective.theta_from_truth</code>	Place a known (<code>pi</code> , <code>A</code> , <code>a</code> , <code>b</code>) in the unconstrained coordinates.
<code>NegativeBinomialHmmObjective</code>	<i>class.</i> Negative log-likelihood of count sequences from a negative binomial HMM.
<code>NegativeBinomialHmmObjective.emissions</code>	The negative binomial family <code>theta</code> 's emission block encodes.
<code>NegativeBinomialHmmObjective.initial</code>	A start that is uninformative but **not** symmetric.
<code>NegativeBinomialHmmObjective.constrain</code>	Split <code>theta</code> into log transitions and the per-state (<code>r</code> , <code>mu</code>).

<code>NegativeBinomialHmmObjective</code>	Place a known truth in the unconstrained coordinates.
<code>theta_from_truth</code>	
<code>forward_log_likelihood</code>	Total log-likelihood of observations by the forward recursion.
<code>align_states</code>	Permutation of fitted hidden states best matching reference .
<code>align_families</code>	Permutation of fitted 's states best matching reference 's.
<code>align_by_key</code>	The permutation minimizing total absolute distance between two key sets.
<code>forward_log_likelihood_from_density</code>	Total log-likelihood from per-site emission scores already computed.
<code>HmmMetrics</code>	<i>class</i> . What an HMM parameter vector means: the forward log-likelihood (issue #778).
<code>EmFit</code>	<i>class</i> . What one Baum-Welch run produced, and what it had to report.
<code>baum_welch</code>	Fit an HMM by expectation-maximization, with no autodiff involved.
<code>baum_welch_family</code>	Baum-Welch over any emission family, with no autodiff involved.

5.8 opt/initialize

Where an optimization starts, as something the caller can choose.

<code>Initializer</code>	<i>class</i> . A source of starting points for one objective.
<code>Initializer.starts</code>	Candidate starting points, in unconstrained coordinates.
<code>FromObjective</code>	<i>class</i> . The objective's own <code>initial()</code> , unchanged.
<code>FromObjective.starts</code>	The single point the objective nominates.
<code>Perturbed</code>	<i>class</i> . The objective's start, tilted by a fixed amount along each coordinate.
<code>Perturbed.starts</code>	One start, off the objective's own by a fixed alternating tilt.
<code>RandomRestart</code>	<i>class</i> . Several starts, drawn around the objective's own.
<code>RandomRestart.starts</code>	<code>n_starts</code> points, the first being the nominal one if asked.
<code>quantile_locations</code>	Evenly spaced quantiles of values , one per state or component.

5.9 opt/mixture

A Gaussian mixture: the reference instance whose start reads the data.

<code>GaussianMixtureObjective</code>	<i>class</i> . Negative log-likelihood of independent observations from a mixture.
<code>GaussianMixtureObjective.n_components</code>	Components in the mixture.
<code>GaussianMixtureObjective.n_channels</code>	Entries an observation carries; 1 for a scalar observation.
<code>GaussianMixtureObjective.observations</code>	The observations being fitted, of the shape they were given in.
<code>GaussianMixtureObjective.variance_floor</code>	The floor the EM oracle refuses at, derived from these observations.
<code>GaussianMixtureObjective.n_parameters</code>	(<code>k</code> - 1) free weights, then <code>k</code> means and <code>k</code> log scales, per channel.

<code>GaussianMixtureObjective.components</code>	The component family <code>theta</code> encodes, differentiable in <code>theta</code> .
<code>GaussianMixtureObjective.initial</code>	Uniform weights, means at quantiles of the data, pooled scales.
<code>GaussianMixtureObjective.theta_from_centres</code>	A start whose component means are <code>centres</code> .
<code>GaussianMixtureObjective.constrain</code>	The mixing weights as log-probabilities, and the component parameters.
<code>GaussianMixtureObjective.theta_from</code>	The unconstrained vector whose <code>constrain</code> is <code>named</code> .
<code>GaussianMixtureObjective.theta_from_truth</code>	Place a known truth in the unconstrained coordinates.
<code>mixture_log_likelihood</code>	$\sum_i \log \sum_k w_k N(y_i; \mu_k, s_k)$, in log space throughout (<code>eq:mixture</code>).
<code>responsibilities</code>	$P(\text{component} \mid \text{observation})$, shape $(n_samples, n_components)$ (<code>eq:responsibilities</code>).
<code>MixtureMetrics</code>	<i>class</i> . What a mixture parameter vector means: its log-likelihood and its k-means cost (issue #778).
<code>MixtureFit</code>	<i>class</i> . What one expectation-maximization run produced.
<code>expectation_maximization</code>	Fit a mixture by EM, with no autodiff involved.
<code>clustering_cost</code>	The k-means objective: summed squared distance to the nearest centre.
<code>optimal_clustering_cost</code>	The exact optimal k-means cost, by dynamic programming over runs.
<code>kmeans_plus_plus</code>	Seed <code>n_centres</code> centres by D-squared sampling (Arthur & Vassilvitskii, 2007; <code>eq:kmeanspp</code>).
<code>uniform_seeds</code>	Seed by drawing distinct observations uniformly.
<code>seeding_guarantee</code>	8 ($\ln k + 2$), the factor of optimal k-means++ is expected within (<code>eq:kmeanspp-bound</code>).
<code>KMeansPlusPlus</code>	<i>class</i> . Starting points seeded from the data by k-means++.
<code>KMeansPlusPlus.starts</code>	Seed each start by k-means++ on the objective's own observations.
<code>canonical_order</code>	Centres sorted, so a relabelling of the same seeding is one seeding.
<code>emission_mixture_plus_plus</code>	<code>Emission_Mixture++</code> : k-means++ with a family's Bregman divergence as the distance (issue #306).

5.10 opt/objective

The optimization interface, and nothing that knows what is being optimized.

<code>Objective</code>	<i>class</i> . A differentiable scalar objective over an unconstrained parameter vector.
<code>Objective.initial</code>	A starting point in unconstrained coordinates.
<code>Objective.constrain</code>	The named, feasible parameters <code>theta</code> encodes.
<code>Objective.theta_from</code>	The unconstrained vector whose <code>constrain</code> is <code>named</code> .

5.11 opt/potts

A 1-D Potts chain in an external field: reference instance of `Objective`.

<code>squaring_is_cheaper</code>	Whether <code>log_partition_by_squaring</code> does no more work than the recursion.
<code>log_partition_by_recursion</code>	<code>log Z</code> by carrying one message across the chain, site by site.
<code>log_partition_by_squaring</code>	<code>log Z</code> with the transfer product reassociated by repeated squaring.
<code>log_partition</code>	Exact <code>log Z</code> for a chain of <code>length</code> sites, by transfer matrix.
<code>graph_statistics</code>	Enumerate every configuration's sufficient statistics for a Potts graph.
<code>log_partition_graph</code>	Exact <code>log Z</code> for a Potts graph, from enumerated statistics.
<code>PottsLatticeObjective</code>	<i>class.</i> Negative log-likelihood of Potts configurations on a graph.
<code>PottsObjective</code>	<i>class.</i> Negative log-likelihood of Potts chains, as an <code>Objective</code> .

5.12 opt/testfunctions

Standard continuous test functions, as `Objective`.

<code>Rosenbrock</code>	<i>class.</i> <code>sum_i b (x_{i+1} - x_i^2)^2 + (a - x_i)^2</code> , minimized at <code>(a, ..., a)</code> (<code>eq:rosenbrock</code>).
<code>Rosenbrock.initial</code>	<i>No docstring.</i>
<code>Rosenbrock.constrain</code>	<i>No docstring.</i>
<code>Rosenbrock.theta_from</code>	<i>No docstring.</i>
<code>Rosenbrock.minimizer</code>	The analytic minimizer, <code>(a, ..., a)</code> , where the value is 0.
<code>Rosenbrock.gradient</code>	The closed-form gradient, for checking autodiff against.
<code>Rastrigin</code>	<i>class.</i> <code>10 n + sum_i (x_i^2 - 10 cos(2 pi x_i))</code> , minimized at the origin (<code>eq:rastrigin</code>).
<code>Rastrigin.initial</code>	<i>No docstring.</i>
<code>Rastrigin.constrain</code>	<i>No docstring.</i>
<code>Rastrigin.theta_from</code>	<i>No docstring.</i>
<code>Rastrigin.minimizer</code>	The analytic global minimizer, the origin, where the value is 0.
<code>Rastrigin.gradient</code>	The closed-form gradient.
<code>Himmelblau</code>	<i>class.</i> <code>(x^2 + y - 11)^2 + (x + y^2 - 7)^2</code> , with four equal global minima (<code>eq:himmelblau</code>).
<code>Himmelblau.initial</code>	<i>No docstring.</i>
<code>Himmelblau.constrain</code>	<i>No docstring.</i>
<code>Himmelblau.theta_from</code>	<i>No docstring.</i>
<code>Himmelblau.gradient</code>	The closed-form gradient.
<code>Himmelblau.nearest_minimum</code>	Which of the four minima point landed on, and how far off it is.
<code>TestFunctionMetrics</code>	<i>class.</i> What a point on a test function means: its value, and how far it is from a minimizer (issue #778).
<code>TestFunctionParams</code>	<i>class.</i> One test function as a declared instance: where it starts and where it ends.
<code>TestFunctionParams.objective</code>	The function itself, at this instance's dimension and start.
<code>TestFunctionSuite</code>	<i>class.</i> The functions a fixture declares, and the knobs a study of them shares.
<code>TestFunctionSuite.named</code>	The functions by name, in the file's order.

<code>TestFunctionSuite.from_ declared</code>	Build the truth from a continuous-test-function fixture's declared mapping.
---	---

6 qa/

6.1 qa/__init__

Cross-cutting quality-assurance figures and tables. See CLAUDE.md in this directory.

No public names.

6.2 qa/backend_agreement

QA figure: the backends against the oracle, and against each other.

<code>agreement</code>	Relative deviation of each backend from brute force, per site count.
<code>build_figure</code>	Assemble the figure and its caption.
<code>main</code>	Render the figure from the command line.

6.3 qa/build

Regenerate QA figures, selected by what the documents cite.

<code>UncitedFigureError</code>	<i>class</i> . Raised when the document cites a figure the manifest cannot render.
<code>selected</code>	Choose the figures to regenerate.
<code>render</code>	Run one figure's script, writing into <code>output_dir</code> .
<code>compare</code>	Find every rebuilt file that differs from its committed counterpart.
<code>main</code>	Regenerate or check the selected QA figures.

6.4 qa/coupled_labelling

QA figure: the coupled model's labelling, planted and recovered, on the lattice.

<code>lattice_shape</code>	The rows and columns of the lattice the figure draws.
<code>lattice_coordinates</code>	Node coordinates read off the lattice, row zero at the top.
<code>planted_labelling</code>	The two-stripe planting the suite and the notebook plant on this lattice.
<code>best_permutation</code>	The renaming of the fit's classes that agrees with <code>planted</code> most.
<code>Recovery</code>	<i>class</i> . One replicate: what was drawn, what was fitted, and how far apart they are.
<code>Recovery.margin</code>	The first planted class's cost less the second's, one per node.
<code>Recovery.field_labelling</code>	The labelling the field alone gives: each node at its cheaper class.
<code>recover</code>	Draw one replicate and fit it from the annealed start.
<code>build_figure</code>	Assemble the three panels and the caption.
<code>main</code>	Render the figure from the command line.

6.5 qa/figure

Shared QA figure/caption writing, so every QA script formats consistently.

<code>latex_integer</code>	Format an integer with underscore separators, escaped for LaTeX.
<code>check_latex_safe</code>	Raise if <code>text</code> contains an unescaped LaTeX special character.
<code>pearson_correlation</code>	Linear correlation of two equal-length samples.
<code>state_label</code>	Render a simulated state as a nucleotide letter when <code>k == 4</code> .
<code>QATable</code>	<i>class.</i> A rendered LaTeX table fragment together with its caption.
<code>write_qa_table</code>	Write a LaTeX <code>tabular</code> fragment and its caption under <code>output_dir</code> .
<code>QAFigure</code>	<i>class.</i> A rendered QA figure together with its caption.
<code>write_qa_figure</code>	Write <code>fig</code> and <code>caption</code> under <code>output_dir</code> , named from <code>stem</code> .

6.6 qa/frustrated_lattices

QA figure: the two frustrated instances whose answer comes from outside.

<code>lattice_layout</code>	Sheared coordinates putting every lattice bond at unit length.
<code>edge_segments</code>	One drawn segment per edge, a wrapped edge ending at a ghost of its target.
<code>ground_state</code>	One enumerated ground state, and how many of its edges agree.
<code>GlassScan</code>	<i>class.</i> The planted energy against descent, per frustration.
<code>glass_scan</code>	Build one instance per declared frustration and descend on each.
<code>build_figure</code>	Assemble the two panels and the caption.
<code>main</code>	Render the figure from the command line.

6.7 qa/hmc_warmup

QA figure: what a Hamiltonian Monte Carlo warm-up buys, and what it costs.

<code>CorrelatedGaussian</code>	<i>class.</i> $-\log N(\text{mean}, \text{covariance})$ up to a constant, over an unconstrained vector.
<code>CorrelatedGaussian.precision</code>	The inverse covariance, which is also this objective's Hessian.
<code>CorrelatedGaussian.initial</code>	A start away from the mode, identical for both chains.
<code>CorrelatedGaussian.constrain</code>	The named parameters; the coordinates are already unconstrained.
<code>CorrelatedGaussian.theta_</code>	The unconstrained vector behind <code>named</code> .
<code>from</code>	
<code>target</code>	The declared target.
<code>chains</code>	Draw the adapted chain and the fixed-parameter chain.
<code>running_mean</code>	The running mean of one coordinate, draw by draw.

<code>settling_draw</code>	The first draw after which the running mean stays within SETTLED .
<code>build_figure</code>	Draw the two panels and write the caption.
<code>main</code>	Render the figure from the command line.

6.8 qa/layout

Shared figure layouts: the compositions more than one figure needs, built once (issue #312).

<code>marker_size</code>	Scatter marker area that shrinks with point count, clamped to <code>[0.1, 25]</code> .
<code>spatial_grid</code>	One panel per feature over shared 2-D coordinates, zeros drawn hollow, a colourbar per panel.
<code>grouped_tracks</code>	<code>n_groups * tracks_per_group</code> axes stacked vertically, a gap of <code>TRACK_GAP</code> between groups.
<code>format_track_axis</code>	Label, limits and gridlines a track shares with its neighbours, so a column of tracks reads as one.
<code>joint_distribution</code>	A scatter with an outlined marginal histogram per group above and to the right.
<code>discrete_legend</code>	A legend of round markers naming <code>categories</code> by <code>colors</code> , placed outside the axes by default.

6.9 qa/likelihood.footprint

QA table: the memory footprint of a likelihood evaluation, against the declared scale.

<code>caterpillar</code>	The deepest unrooted topology on <code>n_taxa</code> leaves, branch lengths unset.
<code>simulation_bytes</code>	What the simulator holds: every node's states, leaf and internal alike.
<code>evaluation_bytes</code>	What pruning holds on a caterpillar: one <code>(n_sites, k)</code> partial per node.
<code>measure</code>	Peak bytes the allocator counts, simulating and then evaluating.
<code>warm_up</code>	Discard one measurement, so the next is a property of the problem size.
<code>render_footprint</code>	Build the LaTeX <code>tabular</code> : the measured sizes, then the declared maximum.
<code>build_caption</code>	Caption text for the footprint table.
<code>build_table</code>	Assemble the <code>tabular</code> body and its caption.
<code>main</code>	Render the footprint table.

6.10 qa/manifest

Which QA figures exist, and what renders each one.

<code>FigureSpec</code>	<i>class</i> . One QA output, and the command that renders it.
<code>FigureSpec.command</code>	Build the argument vector that renders this figure.
<code>cited_stems</code>	Find the figure stems the given LaTeX sources refer to, as one set.
<code>select</code>	Pick the manifest entries rendering <code>stems</code> , in manifest order.
<code>unknown_stems</code>	Find which of <code>stems</code> this manifest cannot render.

6.11 qa/mixture_seeding

QA figure: the Gaussian mixture, its fit, and what seeding buys.

<code>SeedingRatios</code>	<i>class</i> . Seeding cost over the optimal cost, per strategy.
<code>seeding_ratios</code>	Seed the observations <code>SEEDINGS</code> times each way and cost every seeding.
<code>fitted</code>	Expectation-maximization from one <code>k-means++</code> start drawn from <code>rng</code> .
<code>density</code>	The mixture density on <code>grid</code> , from the family's own log-density.
<code>build_figure</code>	Assemble the two panels and the caption.
<code>main</code>	Render the figure from the command line.

6.12 qa/opt_branch_recovery

QA figure: branch-length recovery, and the root pair that cannot be recovered.

<code>recovery</code>	Fit one fixture's branch lengths and return truth, estimate, error, coverage.
<code>split_profile</code>	Log-likelihood as mass moves between two sibling branches at fixed sum.
<code>build_figure</code>	Assemble the two-panel figure and its caption.
<code>main</code>	Render the figure from the command line.

6.13 qa/opt_coverage

QA figure: do the fitted intervals cover the truth at their nominal rate?

<code>potts_coverage</code>	Count covering intervals over independent Potts datasets.
<code>hmm_coverage</code>	Count covering intervals over independent HMM datasets.
<code>build_figure</code>	Assemble the coverage figure and its caption.
<code>main</code>	Render the figure from the command line.

6.14 qa/opt_model_recovery

QA figure: recovering a substitution model, and not inventing one.

<code>fit_model</code>	Simulate under a model, fit the general one, and return the estimates.
<code>truth_vector</code>	The free exchangeabilities, rescaled so the last is 1, then <code>pi</code> .
<code>build_figure</code>	Assemble the two-panel figure and its caption.
<code>main</code>	Render the figure from the command line.

6.15 qa/opt_recovery

QA figure: parameter recovery for both reference instances of `Objective`.

<code>potts_recovery</code>	Fit the Potts fixture and return truth, estimate, error and coverage.
<code>hmm_recovery</code>	Fit the HMM fixture and return truth, estimate, error and coverage.
<code>build_figure</code>	Assemble the two-panel recovery figure and its caption.
<code>main</code>	Render the figure from the command line.

6.16 qa/optimizer_landscapes

QA figure: the three test functions, and where a multi-start fit lands.

<code>objectives</code>	The declared functions, in the fixture's order.
<code>known_minimizers</code>	The published minimizers of one function, shape <code>(m, dimension)</code> .
<code>surface</code>	The objective evaluated on the function's declared drawing grid.
<code>endpoints</code>	Fit every declared function from its declared random restarts.
<code>reached</code>	How many of the fits ended within <code>at_minimum</code> of a global minimizer.
<code>build_figure</code>	Assemble the three panels and the caption.
<code>main</code>	Render the figure from the command line.

6.17 qa/parsimony_zones

QA figure: parsimony where it is wrong by theorem, and where it is right.

<code>zone_tree</code>	<code>((A,B),(C,D))</code> with the four pendant branch lengths given.
<code>zone_gaps</code>	Per-site Fitch gap, long-branch grouping minus generating tree, per zone.
<code>Ranking</code>	<i>class</i> . Every topology's Fitch score on the fixture, and where the search landed.
<code>ranking</code>	Score every topology on the fixture and climb from a random start.
<code>build_figure</code>	Assemble the two panels and the caption.
<code>main</code>	Render the figure from the command line.

6.18 qa/rl_reward_surface

QA figure: the cheap reward surface against the one it stands in for.

<code>mean_branch_length</code>	The generating tree's mean branch length.
<code>reward_surfaces</code>	Score every topology both ways, and sweep the cheap surface's parameter.
<code>build_figure</code>	Assemble the two-panel figure and its caption.
<code>main</code>	Render the figure from the command line.

6.19 qa/rl_tree_policy

QA figure: what a learned policy can and cannot do on a rugged tree environment.

<code>measure</code>	Run the comparison this figure reports.
<code>build_figure</code>	Assemble the two-panel figure and its caption.
<code>main</code>	Render the figure from the command line.

6.20 qa/runner

Shared command-line entry point for the QA figure and table scripts.

<code>ParamsArgument</code>	<i>class</i> . One parameters-file argument, and the loader that reads it.
<code>ParamsArgument.dest</code>	The <code>argparse</code> destination the flag parses into.
<code>Option</code>	<i>class</i> . One command-line option that is not a parameters file.
<code>Option.dest</code>	The <code>argparse</code> destination the flag parses into.
<code>figure_main</code>	Render one QA figure from the command line.
<code>table_main</code>	Render one QA table from the command line.

6.21 qa/search_topologies

QA figure: the tree the search found, beside the best one it rejected.

<code>found_and_runner_up</code>	Fit the search's tree and the best topology it rejected.
<code>build_figure</code>	Assemble the side-by-side comparison and its caption.
<code>main</code>	Render the figure from the command line.

6.22 qa/search_trajectory

QA figure: what hill climbing does, and what it was choosing between.

<code>search_trajectories</code>	Run the searches and the exhaustive sweep this figure reports.
<code>build_figure</code>	Assemble the two-panel figure and its caption.
<code>main</code>	Render the figure from the command line.

6.23 qa/sim_example

QA figure: a worked simulation example – Newick string and alignment.

<code>display_newick</code>	The topology as a <code>mathtext</code> string, for display rather than for parsing.
<code>render_sim_example</code>	Draw <code>dataset</code> 's Newick string and a sliced alignment table on <code>ax</code> .
<code>build_caption</code>	Caption text for the worked-example figure.
<code>build_figure</code>	Render the worked-example figure and its caption from loaded params.
<code>main</code>	Render the figure from the command line.

6.24 qa/sim_problem_sizes

QA table: problem-size parameters across the simulation fixtures.

<code>render_problem_sizes</code>	Build a LaTeX <code>tabular</code> of problem-size parameters, one row per fixture.
<code>build_caption</code>	Caption text for the problem-sizes table.
<code>build_table</code>	Build the problem-sizes <code>tabular</code> body and its caption.
<code>main</code>	Render the table from the command line.

6.25 qa/sim_tree

QA figure: the assumed simulation tree, topology and branch lengths.

<code>tree_layout</code>	Depth (x) and vertical order (y) for every node in <code>tau</code> .
<code>render_sim_tree</code>	Draw <code>tau</code> as a labelled phylogram on <code>ax</code> .
<code>build_caption</code>	Caption text for the simulation-tree figure.
<code>build_figure</code>	Render the simulation-tree figure and its caption from loaded params.
<code>main</code>	Render the figure from the command line.

6.26 qa/style

Shared figure style for the documents.

<code>series_style</code>	Colour, marker and linestyle for series <code>index</code> .
<code>blend_with_white</code>	<code>color</code> mixed with white: <code>alpha = 1</code> is the colour, <code>alpha = 0</code> is white.
<code>with_opacity</code>	One RGBA row per colour, its alpha the matching opacity clipped to <code>[0, 1]</code> .
<code>discrete_palette</code>	State index to hex colour from <code>STATE.PALETTE</code> , with one state optionally drawn in <code>highlight_color</code> .
<code>notebook_style</code>	The letter style at screen resolution, for the notebooks under <code>docs/nb/</code> .
<code>letter_style</code>	Draw inside the documents' figure style.
<code>tolerance_note</code>	State the largest deviation against the tolerance it was checked at.

6.27 qa/tanner_graph

QA figure: the Tanner graph of the enumerable low-density parity-check instance.

<code>TannerLayout</code>	<i>class</i> . Where every node and every edge of the Tanner graph is drawn.
<code>tanner_layout</code>	Lay the code out as two rows of nodes and one segment per nonzero.
<code>build_figure</code>	Draw the Tanner graph and the matrix it comes from, and caption them.
<code>main</code>	Render the figure from the command line.

6.28 qa/topology_accuracy

QA figure: how much data the search needs to recover the true topology.

<code>accuracy</code>	Normalized RF distance from the inferred topology to the truth.
<code>build_figure</code>	Assemble the figure and its caption.
<code>main</code>	Render the figure from the command line.

6.29 qa/tropical_relaxation

QA figure: the tropical Grassmannian relaxation, and the surface it relaxes.

<code>Sweep</code>	<i>class</i> . Panel (a): the measured corner gap and its bound, per temperature.
<code>Surfaces</code>	<i>class</i> . Panel (b): every topology scored on both surfaces.
<code>measure</code>	Fit the quartet table once and take both panels from it.

<code>build_figure</code>	Assemble the two panels and the caption.
<code>main</code>	Render the figure from the command line.

6.30 qa/turbo_waterfall

QA figure: the turbo code's bit error rate against E_b / N_0 .

<code>measure</code>	Every declared point of the instance, under the fixture's own seed.
<code>build_figure</code>	Draw the waterfall and the frame error rate, and caption them.
<code>main</code>	Render the figure from the command line.

7 sample/

7.1 sample/___init___

Samplers, annealers and tempering. See CLAUDE.md in this directory.
No public names.

7.2 sample/accept

The Metropolis accept step, in one place (issue #857).

<code>accept</code>	Accept with probability <code>min(1, exp(log_ratio))</code> , drawing at most one uniform.
<code>accept_drawn</code>	<code>accept</code> with the uniform behind a call.
<code>accept_with</code>	<code>accept</code> on a uniform the caller has already drawn.
<code>accept_ratio</code>	Accept on a ratio the caller has already exponentiated.
<code>acceptance_probability</code>	<code>min(1, ratio)</code> , and zero where the ratio is <code>nan</code> .
<code>accept_at</code>	<code>accept</code> at inverse temperature <code>beta</code> , with the <code>beta = inf</code> limit exact.
<code>accept_at_temperature</code>	<code>accept_at</code> read in temperature, with the uniform behind a call.

7.3 sample/annealed

$\log Z$ by annealing: importance sampling, a resampled population, and the tempering that reads their rungs back.

<code>Resampling</code>	<i>class</i> . How a population is resampled by its weights at a rung.
<code>LogPartition</code>	<i>class</i> . $\log Z$ from an annealed run, and what the weights behind it were worth.
<code>geometric_betas</code>	A ladder of inverse temperatures: exactly 0, then geometric to <code>beta</code> .
<code>annealed_importance_sampling</code>	$\log Z$ from independent annealing runs, weighted by what each one cost (Neal 2001).
<code>population_annealing</code>	$\log Z$ from a population resampled at every rung (Hukushima & Iba 2003; Machta 2010).
<code>SimulatedTempered</code>	<i>class</i> . What one simulated-tempering walker did, at which rung it did it.

<code>SimulatedTempered.</code>	The rung trace as one walker's column, shape <code>(n_recorded,</code>
<code>walkers</code>	<code>1)</code> .
<code>rung_weights</code>	The <code>g_k = -log Z_k</code> a pilot run leaves for
	<code>simulated_tempering</code> .
<code>simulated_tempering</code>	One walker over the ladder, with the rung as a sampled
	variable (Marinari & Parisi 1992).

7.4 sample/balanced

Locally balanced proposals over single-variable changes, and their accept step.

<code>BalancingFunction</code>	<i>class.</i> Which <code>g</code> weights the neighbourhood.
<code>log_ratios</code>	<code>log pi(s') - log pi(s)</code> for every single-variable change
	of state.
<code>log_balanced_weights</code>	<code>log g(pi(s') / pi(s))</code> for every single-variable change.
<code>log_normalizer</code>	<code>log Z(s)</code> : the log of the neighbourhood's total weight.
<code>draw_change</code>	One (variable, value) drawn proportional to
	<code>exp(log_weights)</code> .
<code>log_metropolis_ratio</code>	log of the Metropolis-Hastings ratio of one single-variable
	change.

7.5 sample/gibbs

One Gibbs sampler and one annealer for every problem, over the factor graph (issue #309).

<code>GibbsMove</code>	<i>class.</i> Which single-variable move a chain over the factor
	graph proposes from.
<code>GibbsChain</code>	<i>class.</i> What a run of the sampler returns.
<code>Annealed</code>	<i>class.</i> What annealing returns: the best state visited, and
	the path there.
<code>gibbs_sweep</code>	One heat-bath update of every variable in graph order, in
	place.
<code>factor_taylor_log_ratios</code>	Gibbs-with-gradients' first-order estimate of every single-
	variable change.
<code>factor_autodiff_log_ratios</code>	<code>factor_taylor_log_ratios</code> from the tape: the definition,
	not the route.
<code>balanced_sweep</code>	<code>n_variables</code> gradient-informed proposals, each Metropolis-
	corrected, in place.
<code>sample_factor_graph</code>	Run one chain of single-variable sweeps and record its
	states.
<code>anneal_factor_graph</code>	Simulated annealing by heat-bath sweeps: one sweep per
	schedule step at that step's temperature.
<code>chain_block_sweep</code>	Redraw a chain-shaped subset of variables from its exact
	conditional, in place.
<code>AnnealedTopology</code>	<i>class.</i> What the topology move returns.
<code>anneal_topology</code>	Metropolis over topologies on the fitted log-likelihood, at
	the schedule's temperature.
<code>cached_topology_score</code>	A scorer that fits a topology once per leaf bipartitions and
	reads <code>cache</code> after.
<code>topology_step</code>	One Metropolis step over topologies at <code>temperature</code> .

7.6 sample/hmc

Hamiltonian Monte Carlo over an Objective.

<code>WithGaussianPrior</code>	<i>class.</i> An objective plus an isotropic Gaussian log prior, so it has a posterior.
<code>WithGaussianPrior.</code>	<i>No docstring.</i>
<code>initial</code>	
<code>WithGaussianPrior.</code>	<i>No docstring.</i>
<code>constrain</code>	
<code>WithGaussianPrior.theta_</code>	<i>No docstring.</i>
<code>from</code>	
<code>Kernel</code>	<i>class.</i> One Metropolis transition at a step size, as a warm-up has to see it.
<code>Adaptation</code>	<i>class.</i> A warm-up that sets the step size and the mass diagonal, then stops.
<code>Adapted</code>	<i>class.</i> What a warm-up settled on, so the chain it produced can be reproduced.
<code>HmcChain</code>	<i>class.</i> A chain, and what it cost to get it.
<code>Integrator</code>	<i>class.</i> A symplectic integrator, as the composition of sub-steps it is.
<code>Integrator.force_</code>	Gradient evaluations one trajectory of <code>n_steps</code> costs.
<code>evaluations</code>	
<code>hamiltonian</code>	$U(\text{theta}) + K(\text{momentum})$, the quantity the integrator conserves.
<code>sample</code>	Draw <code>n_samples</code> from the density $\exp(-\text{objective} / \text{temperature})$.
<code>Annealed</code>	<i>class.</i> What one annealing run found, and what it cost.
<code>anneal</code>	Simulated annealing with Hamiltonian proposals: <code>sample</code> on a schedule.
<code>Tempered</code>	<i>class.</i> What one parallel-tempering run found, and what it cost.
<code>parallel_tempering</code>	Replicas at fixed temperatures, exchanging positions by Metropolis.
<code>effective_sample_size</code>	Effective sample size per coordinate, by Geyer's initial positive sequence.

7.7 sample/initialize

Initializers built from a sampler: a chain, an annealing run, a tempering run.

<code>FromChain</code>	<i>class.</i> A start drawn from a short Hamiltonian chain on the objective itself.
<code>FromChain.chain</code>	The chain itself: the draws, the acceptance rate and what it cost.
<code>FromChain.starts</code>	The chain's draws, in the order drawn.
<code>FromAnnealing</code>	<i>class.</i> The best point a Hamiltonian chain visits while its temperature falls.
<code>FromAnnealing.run</code>	The annealing run: its best point, the acceptance rate and its cost.
<code>FromAnnealing.starts</code>	The lowest-valued point visited.

<code>FromTempering</code>	<i>class</i> . The best point a temperature ladder visits, exchanging positions.
<code>FromTempering.run</code>	The tempering run: its best point, both acceptance rates and its cost.
<code>FromTempering.starts</code>	The lowest-valued point visited at any temperature.

7.8 sample/kernels

Compiled kernels whose oracles reproduce them bitwise, beside those oracles.

<code>icm_sweeps</code>	Single-site descent in place; returns the sweeps run.
<code>gibbs_sweep_sites</code>	Heat-bath updates from <code>first</code> on, in place; returns where it stopped.
<code>factor_graph_log_density</code>	<code>sum_f log psi_f</code> at one full assignment, over the edge layout.

7.9 sample/langevin

The Metropolis-adjusted Langevin algorithm: HMC at one leapfrog step.

<code>LangevinChain</code>	<i>class</i> . A Langevin chain, what it cost, and whether it was corrected.
<code>mala</code>	Draw <code>n.samples</code> from <code>exp(-objective / temperature)</code> by Langevin steps.

7.10 sample/potts_keyed

The cluster moves, as something a deterministic `step` can call.

<code>monochrome_partition</code>	Component index per site over the bonds between like-coloured sites.
<code>WolffMove</code>	<i>class</i> . One cluster grown from the action's root, recoloured to its label.
<code>WolffMove.n_nodes</code>	Sites the move expects.
<code>WolffMove.n_states</code>	Labels the move expects.
<code>WolffMove.parametric</code>	A Wolff action names its root and the colour to recolour to.
<code>WolffMove.propose</code>	Grow the cluster at <code>site</code> , recolour it to <code>label</code> , and charge it.
<code>NiedermayerMove</code>	<i>class</i> . One cluster grown under Niedermayer's bond rule, two colours transposed on it.
<code>NiedermayerMove.n_nodes</code>	Sites the move expects.
<code>NiedermayerMove.n_states</code>	Labels the move expects.
<code>NiedermayerMove.parametric</code>	A Niedermayer action names its root and the colour to transpose with.
<code>NiedermayerMove.threshold</code>	E-0, the bond rule's threshold on this lattice.
<code>NiedermayerMove.propose</code>	Grow the cluster at <code>site</code> , transpose its colour with <code>label</code> , charge it.
<code>SwendsenWangMove</code>	<i>class</i> . Every bond drawn, every cluster recoloured, once per action.
<code>SwendsenWangMove.n_nodes</code>	Sites the move expects.
<code>SwendsenWangMove.n_states</code>	Labels the move expects.

<code>SwendsenWangMove.</code>	A Swendsen-Wang action names a temperature and nothing else.
<code>parametric</code>	
<code>SwendsenWangMove.propose</code>	Recolour every cluster once, and charge a sweep.
<code>cluster_moves</code>	Every cluster move on one lattice, keyed as the environment expects them.

7.11 `sample/potts_mcmc`

Monte Carlo move sets on a Potts lattice: single-site, cluster, gradient-informed.

<code>PottsMove</code>	<i>class</i> . Which Monte Carlo move set a chain proposes from.
<code>MoveKind</code>	<i>class</i> . Which move a learned action applies, over the move set <code>PottsMove</code> names.
<code>PottsChain</code>	<i>class</i> . One chain's recorded configurations, and what a sweep cost.
<code>Recolour</code>	<i>class</i> . What one cluster recolouring proposed, and whether the field accepted it.
<code>ClusterCounter</code>	<i>class</i> . Cluster sizes and field acceptances, accumulated over a run.
<code>ClusterCounter.record</code>	Add one recoloured cluster.
<code>ClusterCounter.accept_rate</code>	Accepted over proposed; <code>nan</code> where nothing was proposed.
<code>ClusterCounter.mean_size</code>	Sites per recoloured cluster; <code>nan</code> where none was built.
<code>ClusterCounter.max_size</code>	The largest cluster recoloured; zero where none was built.
<code>ClusterCounter.spanning_fraction</code>	Clusters that spanned the lattice, as a fraction of those built.
<code>tempered</code>	The model whose Boltzmann weight at temperature 1 is this one's at <code>temperature</code> .
<code>sample_potts</code>	Run one chain and return the configuration after every sweep.
<code>AnnealedPotts</code>	<i>class</i> . What one annealing run found, and what it cost.
<code>anneal_potts</code>	Simulated annealing by heat-bath sweeps on a temperature schedule.
<code>TemperedChains</code>	<i>class</i> . What a parallel-tempering run produced.
<code>parallel_tempering</code>	Replicas at fixed temperatures, exchanging configurations by Metropolis.
<code>adapt_ladder_potts</code>	A ladder for <code>parallel_tempering</code> , from its own exchange acceptances.
<code>sample_potts_pair</code>	Two replicas at one temperature, joined by Houdayer's isoenergetic move.
<code>taylor_log_ratios</code>	Gibbs-with-gradients' first-order estimate of every single-flip change.
<code>autodiff_log_ratios</code>	<code>taylor_log_ratios</code> from the tape: the definition, not the route.
<code>niedermayer_threshold</code>	Niedermayer's E.0 for a graph, in the energy units of <code>energies</code> .
<code>houdayer_cluster</code>	Component index per site over the bonds joining two sites where the replicas disagree.

7.12 `sample/schedule`

Temperature schedules: a temperature per step, and nothing about what is tempered.

<code>TempSchedule</code>	<i>class.</i> A positive temperature for each of <code>n_steps</code> steps.
<code>ConstantTempSchedule</code>	<i>class.</i> One temperature throughout. <code>ConstantLR</code> with factor 1.
<code>LinearTempSchedule</code>	<i>class.</i> Straight line from <code>start</code> to <code>end</code> . <code>LinearLR</code> with declared ends.
<code>ExponentialTempSchedule</code>	<i>class.</i> Geometric from <code>start</code> to <code>end</code> . <code>ExponentialLR</code> with a declared end.
<code>CosineTempSchedule</code>	<i>class.</i> Half a cosine from <code>start</code> to <code>end</code> . <code>CosineAnnealingLR</code> with <code>T_max = n_steps - 1</code> .
<code>LadderTempSchedule</code>	<i>class.</i> The temperatures as given, one per step: a ladder is a schedule indexed by rung.
<code>LadderTempSchedule.n_steps</code>	One step per rung.
<code>ladder</code>	A tuple of temperatures from a schedule or from a sequence, the same floats either way.
<code>temperatures</code>	Every temperature of <code>schedule</code> , in step order.
<code>AdaptedLadder</code>	<i>class.</i> What a ladder warm-up settled on, and what it measured there.
<code>adapt_ladder</code>	Insert and remove temperatures until every neighbouring pair exchanges within <code>band</code> .
<code>FeedbackLadder</code>	<i>class.</i> What a round-trip warm-up placed, and the circulation it placed it from.
<code>adapt_ladder_by_round_trips</code>	Redistribute a ladder of fixed length so a walker's round trip is fastest.

7.13 sample/slice

Slice sampling over an `Objective`.

<code>SliceDirection</code>	<i>class.</i> The line a univariate update runs along.
<code>SliceUpdate</code>	<i>class.</i> One univariate update along one line, and what it spent.
<code>SliceChain</code>	<i>class.</i> A slice-sampled chain, and what it cost.
<code>slice_sample</code>	Draw <code>n_samples</code> sweeps from the density <code>exp(-objective)</code> .
<code>slice_update</code>	Neal's (2003, §3.1) stepping-out and shrinkage along one line.

7.14 sample/statistics

Three statistics the searches are judged by, and no more.

<code>chi_square_p_value</code>	Tail probability of a chi-square goodness-of-fit statistic.
<code>sign_test_p_value</code>	Exact two-sided sign test on paired differences, ties dropped.
<code>integrated_autocorrelation_time</code>	Sweeps a chain must run to buy one independent sample of series.

7.15 sample/tempered

A tempered ensemble over structures, composed from moves that already exist (issue #331).

<code>TemperedEnsemble</code>	<i>class.</i> What a replica-exchange run over structures produced.
<code>TemperedEnsemble.round_trip_time</code>	Recorded sweeps a walker spends per round trip, averaged over walkers.
<code>TemperedEnsemble.replica_at</code>	The index of the replica at <code>temperature</code> .
<code>round_trips</code>	Round trips completed per walker: the coldest rung to the hottest and back.
<code>round_trip_time</code>	Recorded sweeps a walker spends per round trip, averaged over walkers.
<code>up_fraction</code>	Fraction of labelled visits at each rung made by a walker on its way up.
<code>tempered_factor_graph</code>	Replicas of the heat-bath sweep over <code>graph</code> at fixed temperatures, exchanging by Metropolis.
<code>tempered_potts_pair</code>	A replica pair per rung, exchanging along the ladder, joined by Houdayer's move.
<code>tempered_topologies</code>	Replicas of the Metropolis topology move at fixed temperatures, exchanging by Metropolis.
<code>adapt_ladder_round_trips</code>	A ladder for <code>parallel_tempering</code> , placed by its own round trips.

8 sandbox/

8.1 sandbox/__init__

The conserved home: implementations kept for what they referee, not for a hot path (issue #322).

No public names.

8.2 sandbox/circulant_schedule

A per-step circulant rate, superseded by the matrix forms (issue #658).

<code>circulant_schedule</code>	One circulant per step, from one self-transition rate per step.
---------------------------------	---

8.3 sandbox/count_emissions

The count families as they stood before they took a covariate (issue #631).

<code>NegativeBinomialEmission</code>	<i>class.</i> A count drawn from <code>NegativeBinomial(dispersion[state], mean[state])</code> .
<code>NegativeBinomialEmission.from_probability</code>	Build from the textbook (r, p) , where $\mu = r (1 - p) / p$.
<code>NegativeBinomialEmission.n_states</code>	Hidden states this family emits from.
<code>NegativeBinomialEmission.is_discrete</code>	True: the support is the non-negative integers.
<code>NegativeBinomialEmission.observation_dtype</code>	Floating point, though the support is integral.
<code>NegativeBinomialEmission.dispersion</code>	Per-state <code>r</code> , shape <code>(n_states,)</code> .

NegativeBinomialEmission.	Per-state μ , shape (n_states,).
mean	
NegativeBinomialEmission.	$\mu + \mu^2 / r$, the relation that makes this a count
variance	model.
NegativeBinomialEmission.	Draw one count per entry of states.
sample	
NegativeBinomialEmission.	The negative binomial log-probability of every count under
log_density	every state.
NegativeBinomialEmission.	$r \log((r + \mu) / (r + y)) + y \log(y (r + \mu) /$
bregman_divergence	$(\mu (r + y)))$.
NegativeBinomialEmission.	Raise if an observation is not a non-negative integer.
validate	
NegativeBinomialEmission.	The M step: a closed form for the mean, and a solve for the
reestimate	dispersion.
NegativeBinomialEmission.	The per-state mean and variance , both in observation
alignment_key	units.
NegativeBinomialEmission.	dispersion and mean, the parameters the model is stated
named_parameters	in.
BetaBinomialEmission	<i>class</i> . A count of successes in trials[state] attempts
	with a random rate.
BetaBinomialEmission.n_	Hidden states this family emits from.
states	
BetaBinomialEmission.is_	True: the support is {0, ..., n}.
discrete	
BetaBinomialEmission.	Floating point, since lgamma consumes the counts.
observation_dtype	
BetaBinomialEmission.	Per-state n, shape (n_states,).
trials	
BetaBinomialEmission.	Per-state a, shape (n_states,).
alpha	
BetaBinomialEmission.	Per-state b, shape (n_states,).
beta	
BetaBinomialEmission.	a + b: how close this is to a binomial. Larger is closer.
concentration	
BetaBinomialEmission.	$n a / (a + b)$.
mean	
BetaBinomialEmission.	$n p (1 - p) (n + a + b) / (1 + a + b)$, above the bi-
variance	nomial's.
BetaBinomialEmission.	Draw a rate from the Beta, then a binomial count at that
sample	rate.
BetaBinomialEmission.log_	$\log C(n, y) + \log B(y + a, n - y + b) - \log B(a,$
density	$b)$.
BetaBinomialEmission.	The log-density gap to the best rate at this concentration.
bregman_divergence	
BetaBinomialEmission.	Raise if an observation is not an integer in [0,
validate	$\max(\text{trials})]$.
BetaBinomialEmission.	Alternating bisection for (a, b); see
reestimate	<code>_solve_beta_binomial</code> .
BetaBinomialEmission.	The per-state mean and variance, both in observation units.
alignment_key	

<code>BetaBinomialEmission.</code>	<code>alpha</code> and <code>beta</code> . The trial count is a constant.
<code>named_parameters</code>	
<code>identifiable_concentration_bound</code>	The concentration above which this much data cannot tell <code>a + b</code> from infinity.
<code>identifiable_dispersion_bound</code>	The dispersion above which this much data cannot tell <code>r</code> from infinity.

8.4 `sandbox/maxflow_declined`

The three max-flow kernels issue #715 declined, behind the `sandbox` feature.

<code>DeclinedKernel</code>	<i>class</i> . The kernels issue #715 measured and did not keep.
<code>min_cut</code>	The minimum cut by a declined kernel; the package's <code>min_cut</code> otherwise.
<code>ising_ground_state</code>	The exact two-state ground state by a declined kernel.

8.5 `sandbox/polar_reference`

Successive cancellation, written the way the textbook states it.

<code>decode_sc_reference</code>	The source vector plain successive cancellation decides, all <code>N</code> positions.
----------------------------------	--

8.6 `sandbox/pruning_burn`

Felsenstein pruning taped by `burn`, behind one `torch.autograd.Function`.

<code>log_likelihood</code>	Total log-likelihood, differentiable w.r.t. <code>branch_lengths</code> .
-----------------------------	---

8.7 `sandbox/rectangular_hmm`

The rectangular Baum-Welch, conserved as the referee of the ragged one.

<code>baum_welch_rectangular</code>	Baum-Welch over any emission family, with no autodiff involved.
-------------------------------------	---

8.8 `sandbox/region_graph`

Regions of variables, their counting numbers, and the Kikuchi free energy.

<code>Region</code>	<i>class</i> . A set of variables, the factors it owns, and how often it is counted.
<code>RegionGraph</code>	<i>class</i> . Regions over one factor graph, ordered by inclusion.
<code>RegionGraph.counts</code>	How often each variable is counted, which a valid graph makes one.
<code>RegionGraph.factor_counts</code>	How often each factor's energy is counted.
<code>RegionGraph.check</code>	Refuse a region graph that counts anything other than once.
<code>region_graph</code>	The cluster-variation region graph over <code>clusters</code> , closed and counted.
<code>bethe_region_graph</code>	The two-layer region graph whose free energy <i>is</i> the Bethe one.
<code>kikuchi_free_energy</code>	F_K: the region-based free energy of a set of beliefs.
<code>lattice_plaquettes</code>	The unit cells of a 2-D square lattice, as clusters of variable names.

<code>RegionBeliefs</code>	<i>class</i> . What generalized belief propagation returns, from a run that converged.
<code>generalized_belief_propagation</code>	Parent-to-child messages to a stationary point of the Kikuchi energy.

8.9 sandbox/tropical

The tropical Grassmannian relaxation: tree topologies as a smooth objective (issue #408).

<code>quartet_indices</code>	Every quartet of taxon indices, ascending.
<code>pairing_positions</code>	Condensed-vector positions of the six pairs of every quartet.
<code>QuartetTable</code>	<i>class</i> . A score per quartet per resolution, and the taxa it is indexed by.
<code>QuartetTable.n_taxa</code>	Taxa the table covers.
<code>QuartetTable.n_quartets</code>	Quartets in the table.
<code>QuartetTable.ranges</code>	Per quartet, the spread of its three scores.
<code>quartet_table</code>	Fit all three resolutions of every quartet.
<code>resolutions</code>	Which of <code>PAIRINGS</code> a topology induces on each quartet.
<code>discrete_score</code>	$D(T)$: the topology's quartet score, summed over quartets.
<code>corner</code>	A tree metric of topology : the relaxation's corner for that tree.
<code>condensed</code>	A square distance matrix as the condensed vector this module optimizes.
<code>expanded</code>	A condensed vector back as a symmetric (n, n) matrix with a zero diagonal.
<code>relaxed_score</code>	$F_{\tau}(d)$: the quartet score under softmax weights, differentiable in d .
<code>corner_gaps</code>	Per quartet, the gap between the smallest pairing sum and the next.
<code>corner_bound</code>	How far F_{τ} may sit from D at a tree metric.
<code>temperature_for</code>	The largest τ <code>corner_bound</code> certifies against <code>tolerance</code> .
<code>TropicalOptimum</code>	<i>class</i> . What gradient ascent on F_{τ} reached.
<code>anneal</code>	Geometric temperature schedule, evaluated at one step.
<code>optimize</code>	Gradient ascent on F_{τ} from a starting metric, then read the tree off.
<code>metric_from_split_weights</code>	The tree metric a set of weighted splits implies, as a square matrix.

9 scripts/

9.1 scripts/__init__

Command-line entry points; `run_snakes_and_ladders` is the one console script.
No public names.

9.2 scripts/run_snakes_and_ladders

Command-line entry point for `snakes_and_ladders`.

<code>main</code>	Print a placeholder message until the CLI exists.
-------------------	---

10 search/

10.1 search/__init__

Move sets, samplers and exact solvers. See CLAUDE.md in this directory.

No public names.

10.2 search/alpha_expansion

Alpha expansion: approximate MAP on a Potts MRF, with a proved bound.

ExpansionResult	<i>class.</i> A labelling, and what reaching it cost.
expand	The optimal alpha -expansion of labelling , by one minimum cut.
alpha_expansion	Cycle over labels until a full sweep lowers nothing.
SweepOrder	<i>class.</i> The order one sweep visits the sites in — a parameter, not a second method.
iterated_conditional_modes	Single-site descent: the baseline alpha expansion has to beat.
swap	The optimal alpha - beta swap of labelling , by one minimum cut.
alpha_beta_swap	Cycle over every label pair until a full sweep lowers nothing.

10.3 search/bifurcation

Simulated bifurcation: a Potts ground state read off a relaxed oscillator per site and label.

BifurcationResult	<i>class.</i> A labelling read off the oscillators, and what it cost.
simulated_bifurcation	A low-energy labelling by ballistic simulated bifurcation.

10.4 search/decoding

Two estimators of a labelling, and which loss each one minimizes.

marginal_decode	The maximum-posterior-marginal labelling: each site at its own argmax.
expected_site_errors	$\sum_i (1 - p_i(y_i))$: how many sites a labelling gets wrong, in mean.

10.5 search/ground_state

Ground-state recovery on a Potts lattice in a per-site field, refereed twice.

Rung	<i>class.</i> One instance of the comparison, and what can referee its energy.
Rung.n_nodes	Sites.
Rung.visits_per_sweep	Site visits one heat-bath sweep costs: a write per site, a read per edge end.
uniform_ground_energy	$-J * n_edges$: the exact optimum of a zero-field ferromagnet.
lattice_rung	One member of the sizing family: a square lattice at its critical coupling.
rung_field	The field and class ladder of a rung at n_states , from one fixture.

Structure	<i>class</i> . What the generating parameters say about a labelling.
quartiles	Which size quartile each site is in, 0 smallest to 3 largest.
structure	Score a labelling against the parameters that built the field.
expansion_bracket	The interval the optimum lies in, from the expansion's factor-2 bound.
MethodRun	<i>class</i> . One method on one rung under one generator.
run_greedy	The field-only labelling: every site takes its own best class, coupling ignored.
run_icm	Iterated conditional modes: single-site descent in index order.
run_gibbs_zero	Gibbs at $T = 0$ in a random sweep order, which is ICM under a permutation.
run_tempering	Parallel tempering over a geometric ladder, charged for every replica.
run_alpha_expansion	Alpha expansion: the only entry carrying a bound, and the bracket's lower end.
run_alpha_beta_swap	Alpha-beta swap: the cheaper move, and the one with no bound.
run_max_product	Max-product on the factor graph: MAP-exact on a tree, and this is not a tree.
run_bifurcation	Simulated bifurcation, one replica, the budget spent in integration steps.
outcome	A Outcome from a run, for compare .
MethodRecord	<i>class</i> . One entry's run, carried out of the comparison on its outcome.
recorded	Every run comparison 's cells made, in cell order.
Entry	<i>class</i> . One named method, in the shape compare calls.
entries	Every entry by name, for compare .

10.6 search/infer

Topology search: the outer loop that the optimizer deliberately excludes.

Inference	<i>class</i> . The outcome of a search.
infer	Hill-climb over topologies, fitting continuous parameters per candidate.
score_topology	Maximized log-likelihood of one topology, with no search.
ParsimonyInference	<i>class</i> . The outcome of a large-parsimony search.
parsimony_search	Hill-climb over topologies on the parsimony score: large parsimony (eq:large-parsimony).

10.7 search/initialize

Data-driven starts for the tree: from distances, and from the Hadamard spectrum.

FromDistances	<i>class</i> . Neighbor joining on pairwise distances, as the start of a fit or a search.
FromDistances.tree	The neighbor-joining tree of the alignment's distances, lengths floored.
FromDistances.starts	One start: the neighbor-joining lengths on the objective's own topology.

<code>FromHadamard</code>	<i>class.</i> The closest tree of the Hadamard conjugation, as the start of a fit or a search.
<code>FromHadamard.tree</code>	The closest tree of the alignment's spectrum, lengths in the model's units.
<code>FromHadamard.starts</code>	One start: the conjugation's split weights on the objective's own topology.

10.8 search/max_cut

Max-Cut, which is the antiferromagnetic Ising ground state under another name.

<code>MaxCutResult</code>	<i>class.</i> A cut, and the certificate that comes with it.
<code>cut_value</code>	Total weight of the edges this assignment separates.
<code>enumerate_max_cut</code>	The true maximum cut, by trying every assignment.
<code>goemans_williamson</code>	Relax to unit vectors, maximize, then round with a random hyperplane.
<code>complete_bipartite</code>	A graph whose maximum cut is every edge, known without solving anything.

10.9 search/maxflow

Max flow, and the exact ground state of a ferromagnetic Ising model.

<code>FlowNetwork</code>	<i>class.</i> A directed graph with capacities, held as paired residual arcs.
<code>FlowNetwork.from_arcs</code>	Every arc at once, in the order <code>add_edge</code> would have appended them.
<code>FlowNetwork.add_edge</code>	Add <code>source -> sink</code> , with <code>reverse</code> capacity on the back arc.
<code>FlowNetwork.as_arrays</code>	The paired arcs as <code>(arcs, capacity, reverse)</code> , one row per edge.
<code>MinCut</code>	<i>class.</i> The value of a maximum flow, and the partition it certifies.
<code>max_flow</code>	Dinic's algorithm: repeated level graphs and blocking flows.
<code>check_non_negative_couplings</code>	Refuse a negative coupling, saying which construction the sign holds up.
<code>ising_ground_state</code>	The exact minimum-energy configuration of a two-state ferromagnet.
<code>cut_energy</code>	The energy a cut of capacity <code>cut_value</code> corresponds to.

10.10 search/maxflow_rust

Rust Boykov–Kolmogorov max flow (`snakes_and_ladders.oxi_snakes_and_ladders`), pinned against `snakes_and_ladders.search.maxflow`.

<code>ising_ground_state</code>	The exact two-state ferromagnetic ground state, computed in Rust.
<code>ising_ground_states</code>	The ground states of a batch of per-node fields on one graph.
<code>min_cut</code>	Maximum flow and the minimum cut it certifies, computed in Rust.

10.11 search/neighbor_joining

Neighbor joining: the tree a distance matrix implies, and whether it is tree-like.

<code>neighbor_joining</code>	The neighbor-joining tree of a distance matrix (Saitou and Nei 1987).
<code>split_lengths</code>	Branch length per split, keyed as <code>branch_splits</code> keys them.
<code>atteson_radius</code>	Half the shortest branch: the error every distance may carry and neighbor joining still recover <code>tau</code> .
<code>four_point_violation</code>	The largest four-point-condition gap over every quartet.

10.12 search/projection

The coupled model in projection, and the seedings its emission parameters start from (issue #541).

<code>ProjectedCounts</code>	<i>class</i> . The coupled instance with its coupling dropped: one mixture, and its truth.
<code>ProjectedCounts.n_components</code>	$M * K$, the components the projection mixes over.
<code>ProjectedCounts.n_samples</code>	Observations drawn.
<code>flatten</code>	The M per-class families as one $M \times K$ family, class-major.
<code>project</code>	Draw <code>n_samples</code> observations from the projection of a coupled model.
<code>CountPairAt</code>	<i>class</i> . Places an <code>IndependentCountPair</code> component on each observed pair.
<code>Seeding</code>	<i>class</i> . What one candidate produced, and what it charged for it.
<code>euclidean_seeding</code>	k-means++ as it stands: D-squared sampling under squared Euclidean distance.
<code>emission_seeding</code>	<code>EmissionMixture++</code> : the same scheme under the family's own Bregman divergence.
<code>data_seeding</code>	Components placed on observations drawn uniformly, without replacement.
<code>prior_seeding</code>	Components drawn from a prior over the observed range, reading no observation.
<code>burn_in_seeding</code>	The coupled model's own start in projection: a short fit on a subsample.
<code>surrogate</code>	The <code>Objective</code> a chain-based candidate samples: a Gaussian mixture on the first channel.
<code>chain_seeding</code>	A short Hamiltonian chain on the surrogate; the last draw is the seeding.
<code>annealed_seeding</code>	The single-chain control for tempering: the best point of a falling temperature.
<code>tempered_seeding</code>	Parallel tempering on the surrogate; the best point at any temperature.
<code>Fitted</code>	<i>class</i> . One seeding, and the fit it started.
<code>Fitted.log_likelihood</code>	The projected log-likelihood the fit reached.
<code>fit_projection</code>	Seed by one candidate, then fit the projected mixture within the budget.
<code>SeededFit</code>	<i>class</i> . One candidate as a <code>snakes_and_ladders.opt.budget</code> method.

10.13 search/spatio_sequential

Fitting the coupled spatio-sequential model: block ascent, three label solvers, and the annealed start (issue #306).

<code>LabelSolver</code>	<i>class.</i> How the label block is solved.
<code>SpatioSequentialFit</code>	<i>class.</i> What block ascent returns.
<code>m_step</code>	Re-estimate every class's emissions, <code>Pi_m</code> and the shared <code>t</code> from the E step.
<code>label_step</code>	One label block: the ground state of the Potts model in the field, by <code>solver</code> .
<code>fit_spatio_sequential</code>	Block-coordinate ascent on $\log p(\mathbf{x}, \mathbf{1} \mid \boldsymbol{\theta})$.
<code>label_accuracy</code>	The fraction of nodes labelled as planted, up to the best permutation of classes.
<code>seed_emissions</code>	<code>EmissionMixture++</code> for every class: seeds under the family's own divergence.
<code>graph_burn_in</code>	<code>GraphBurnIn++</code> (the textbook's burn-in algorithm): the blocks while the inverse temperature rises.

10.14 search/support

How sure a discrete search is of the structure it returns (issues #270, #331).

<code>SupportKind</code>	<i>class.</i> Which set the weight is taken over, and how.
<code>Support</code>	<i>class.</i> What a discrete search reports beside its structure.
<code>TemperedSupport</code>	<i>class.</i> A tempered weight with the diagnostics that decide whether to believe it.
<code>neighbourhood_support</code>	The flat-prior weight of <code>topology</code> among itself and its neighbours under <code>moves</code> .
<code>enumerated_support</code>	The flat-prior weight of <code>topology</code> over every topology on its leaves.
<code>neighbourhood_labelling_support</code>	The flat-prior weight of <code>labelling</code> among itself and every single-site change.
<code>enumerated_labelling_support</code>	The flat-prior weight of <code>labelling</code> over every labelling of <code>graph</code> .
<code>tempered_labelling_support</code>	The fraction of <code>ensemble</code> 's temperature-one replica spent at <code>labelling</code> .
<code>tempered_topology_support</code>	The fraction of <code>ensemble</code> 's temperature-one replica spent at <code>topology</code> .
<code>internal_splits</code>	The bipartitions with at least two leaves on each side: what a bootstrap counts.
<code>bootstrap_support</code>	Felsenstein's bootstrap: per internal split of <code>topology</code> , the fraction of replicates whose search returns it.
<code>split_pattern_support</code>	Fraction of sites compatible with <code>split</code> : at most one state on both sides.
<code>pattern_support</code>	Per internal split of <code>topology</code> , <code>split_pattern_support</code> .

10.15 search/tightening

A dual bound on a Potts ground state, and the plaquettes that tighten it.

<code>Certificate</code>	<i>class.</i> A labelling, an upper bound on the optimum, and the gap between them.
--------------------------	---

<code>Certificate.gap</code>	<code>energy - bound</code> , what is not established. Zero certifies.
<code>Certificate.optimal</code>	Whether the gap has closed to floating-point noise.
<code>dual_bound</code>	Bound the Potts ground-state energy from below, and decode under it.

11 sim/

11.1 sim/__init__

Data generation and ground-truth retention. See CLAUDE.md in this directory.
No public names.

11.2 sim/canonical

Canonical problem instances whose answer is known from outside this suite.

<code>PlantedSpinGlass</code>	<i>class</i> . A +/- J instance and the state planted in it.
<code>frustrated_triangular_lattice</code>	The triangular Ising antiferromagnet: frustration with a closed form.
<code>minimum_frustrated_edges</code>	Edges that must agree in any two-state ground state, exactly (<code>eq:triangular-ground-state</code>).
<code>planted_spin_glass</code>	A +/- J spin glass on a sparse random graph, built around a state (<code>eq:planted-energy</code>).
<code>ambiguous_hmm</code>	A two-state HMM whose Viterbi and posterior decodings differ.
<code>FrustratedLatticeParams</code>	<i>class</i> . The two frustrated instances of one fixture, and the scan over them.
<code> FrustratedLatticeParams.lattice</code>	The declared triangular antiferromagnet.
<code> FrustratedLatticeParams.glass</code>	One planted instance of the scan, at frustration.
<code> FrustratedLatticeParams.from_declared</code>	Build the truth from a frustrated-lattice fixture's declared mapping.

11.3 sim/capacity

Channel capacity, and the largest noise a rate can carry.

<code>binary_entropy</code>	$H(p) = -p \log_2 p - (1-p) \log_2 (1-p)$, in bits.
<code>erasure_capacity</code>	$1 - \text{eps}$: the fraction of symbols that arrive.
<code>symmetric_capacity</code>	$1 - H(p)$, zero at $p = 1/2$ and one at both certainties.
<code>gaussian_capacity</code>	The binary-input Gaussian channel's capacity, by quadrature.
<code>capacity</code>	The capacity of one of the three declared channels.
<code>noise_at_capacity</code>	The noise level whose capacity equals <code>rate</code> : the Shannon limit.
<code>eb_n0_decibels</code>	E_b/N_0 in dB at a noise scale and a rate.
<code>decibels_at_capacity</code>	The binary-input Gaussian limit at <code>rate</code> , in dB.

11.4 sim/convolutional

Convolutional and turbo codes: the shift register, its trellis, and the encoder.

<code>octal_taps</code>	The GF(2) coefficients of an octal generator, constant term first.
<code>Trellis</code>	<i>class</i> . A shift register's state machine, as arrays indexed by (state, input).
<code>Trellis.n_states</code>	2^{**} memory.
<code>recursive_systematic_trellis</code>	The trellis of the rate-1/2 RSC encoder [1, feedforward / feedback].
<code>encode_stream</code>	Run the register over inputs from the zero state.
<code>terminate</code>	message followed by the m tail inputs that empty the register.
<code>random_interleaver</code>	A uniformly random permutation of $0 \dots \text{length} - 1$, drawn from rng .
<code>inverse_permutation</code>	The permutation that undoes permutation .
<code>TurboCode</code>	<i>class</i> . Two registers, one interleaver, and the layout of the transmitted word.
<code>TurboCode.message_length</code>	K.
<code>TurboCode.steps</code>	$K + m$: trellis steps each constituent encoder runs.
<code>TurboCode.block_length</code>	$3K + 4m$: bits transmitted per message.
<code>TurboCode.rate</code>	$K / (3K + 4m)$, the rate the noise variance is set from.
<code>TurboCode.slices</code>	Where each stream sits in the transmitted word.
<code>turbo_code</code>	Build the code: one register, one random interleaver drawn from rng .
<code>turbo_encode</code>	The $3K + 4m$ transmitted bits of message .
<code>parity_check</code>	A parity-check matrix of the turbo code, by nullspace over GF(2).
<code>turbo_generator_matrix</code>	The $(K, 3K + 4m)$ generator: row i encodes the i -th unit message.
<code>TurboParams</code>	<i>class</i> . Fully-specified truth for a turbo-code fixture.
<code>TurboParams.code</code>	Draw the interleaver this fixture declares and build the code.
<code>TurboParams.trellis</code>	The constituent encoder's trellis, without drawing an interleaver.
<code>TurboParams.from_declared</code>	Load and validate a turbo fixture yaml. The generators are written in the file as octal strings ("7", "15") and read here with base 8, which is the notation every reference states them in; reading them as decimal would silently build a different register.

11.5 sim/count_pairs

Two-channel count observations for the coupled model, and the 5K instance (issue #399).

<code>split_covariate</code>	One covariate per channel, from the axis the observation already has.
<code>IndependentCountPair</code>	<i>class</i> . A total and a number of successes, drawn independently, per hidden state.
<code>IndependentCountPair.total</code>	The negative-binomial channel.
<code>IndependentCountPair.successes</code>	The beta-binomial channel.
<code>IndependentCountPair.n_states</code>	Hidden states this family emits from.

<code>IndependentCountPair.is_discrete</code>	True: both channels are counts.
<code>IndependentCountPair.observation_dtype</code>	Floating point, as both channels' <code>lgamma</code> terms require.
<code>IndependentCountPair.sample</code>	Draw one pair per entry of <code>states</code> , shape <code>states.shape + (2,)</code> .
<code>IndependentCountPair.log_density</code>	The pair's log-density under every state: the two channels' sum.
<code>IndependentCountPair.bregman_divergence</code>	The pair's divergence under every state: the two channels' sum.
<code>IndependentCountPair.validate</code>	Raise unless both channels are valid counts for their own family.
<code>IndependentCountPair.reestimate</code>	Each channel's own M step, reported together.
<code>IndependentCountPair.alignment_key</code>	Both channels' means, shape <code>(n_states, 2)</code> : two states emitting the same pair are one state.
<code>IndependentCountPair.named_parameters</code>	Both channels' parameters, each prefixed by its channel.
<code>aggregate</code>	The family a sum of <code>factor</code> consecutive draws is distributed under.
<code>binned_model</code>	The model a bin factor puts the data under, without drawing the data.
<code>BinInstance</code>	<i>class</i> . One of the instances a count-pair fixture declares.
<code>SpatioSequentialCountsParams</code>	<i>class</i> . A declared count-pair instance: the model, its seed, and its bin factors.
<code>SpatioSequentialCountsParams.factors</code>	The declared bin factors, in file order.
<code>SpatioSequentialCountsParams.key_factor</code>	The bin factor of the key instance.
<code>SpatioSequentialCountsParams.marker</code>	The tier the full test at <code>factor</code> runs in.
<code>SpatioSequentialCountsParams.from_declared</code>	Load and validate a count-pair coupled fixture yaml. **The emissions are declared as ladders, not as a table.** <code>M x K</code> families carrying five parameters each is 500 numbers at the declared 5K size, which no reader checks and no reviewer reads. What the file states instead is one ladder per parameter over the <code>K</code> states, shared by every class, and two per-class vectors that separate the classes: the negative binomial's mean is scaled by <code>class_mean_scale[m]</code> and the beta-binomial's rate shifted by <code>class_rate_shift[m]</code> . Everything is still a literal in the file, and the number of them is <code>5 K + 2 M</code> .
<code>CountPairInstance</code>	<i>class</i> . One bin factor's data, with the model it is distributed under.
<code>CountPairInstance.nbytes</code>	Bytes the observations occupy, the term that decides whether an instance fits.
<code>planted_labels</code>	Contiguous bands of rows, one class per band, shape <code>(n_nodes,)</code> .
<code>chain_states</code>	Every class's hidden path, shape <code>(M, S)</code> , as a walk on <code>Z.K</code> .
<code>vertex_covariate</code>	<code>params.covariate</code> for one vertex, as a draw over its <code>S</code> positions wants it.

<code>simulate_count_pairs</code>	Draw the fine instance: the planted labels, every class's chain, and every vertex's counts.
<code>coarsen</code>	Sum both channels over blocks of <code>factor</code> consecutive positions.
<code>counts_digest</code>	A digest of an instance's counts: 16 hex characters of their SHA-256.

11.6 `sim/count_pairs_rust`

The count-pair simulator in Rust, checked against the NumPy one (issue #399).

<code>simulate_count_pairs</code>	Draw the fine instance: the planted labels, every class's chain, and every vertex's counts.
<code>fine_instance</code>	The fine instance a fixture file declares, drawn once and held.
<code>binmed_instance</code>	The instance a fixture file declares at one bin factor.

11.7 `sim/css`

The CSS code a self-orthogonal parity-check matrix defines.

<code>CssCode</code>	<i>class</i> . A CSS code $H_x = H_z = H$, with a basis of its logical operators.
<code>CssCode.n_qubits</code>	The block length <code>n</code> : one qubit per column of H .
<code>CssCode.n_logical</code>	$k = n - 2 \text{rank}(H)$, the encoded qubits.
<code>CssCode.from_parity_check</code>	The code $H_x = H_z = \text{checks}$, with its logical basis built.
<code>CssCode.is_stabilizer</code>	Whether <code>vector</code> lies in the row space of H .
<code>CssCode.logical_label</code>	<code>logical_z v</code> over $\text{GF}(2)$, per row of <code>vectors</code> .
<code>CssCode.four_cycles</code>	The four-cycles of the Tanner graph of H .
<code>sample_x_error</code>	An X error on every qubit independently, and the ratios it gives.
<code>CssBicycleParams</code>	<i>class</i> . Fully-specified truth for a CSS code over a bicycle matrix.
<code>CssBicycleParams.code</code>	Draw the CSS code this fixture declares.
<code>CssBicycleParams.channel</code>	The declared X error channel.
<code>CssBicycleParams.from_declared</code>	Build the truth from a CSS bicycle fixture's declared mapping.

11.8 `sim/elementary_codes`

The codes whose maximum-likelihood decoder is closed form, and CRC.

<code>repetition_code</code>	The $(n, 1)$ code: every bit equals the first.
<code>single_parity_check</code>	The $(n, n-1)$ code: one all-ones row, so every word has even weight.
<code>hamming_code</code>	The perfect $(2^m - 1, 2^m - 1 - m)$ code, or its extended form.
<code>golay_code</code>	The perfect $(23, 12)$ binary Golay code, or its extended $(24, 12)$.
<code>is_perfect</code>	Whether the Hamming spheres of radius <code>t</code> tile the space exactly.
<code>repetition_error_rate</code>	The closed-form block error probability of the $(n, 1)$ code.

<code>crc_remainder</code>	<code>message</code> shifted by the polynomial's degree, modulo it, over GF(2).
----------------------------	---

11.9 `sim/emission_mixture`

Ground truth and data generation for a mixture of count emissions.

<code>EmissionMixtureParams</code>	<i>class</i> . Fully-specified truth for a count-emission mixture fixture.
<code>EmissionMixtureParams.n_components</code>	Components in the mixture.
<code>EmissionMixtureParams.from_declared</code>	Build the truth from a count-emission-mixture fixture's declared mapping.
<code>SimulatedEmissionMixtureDataset</code>	<i>class</i> . A simulated dataset together with the labels and its truth.
<code>simulate_emission_mixture</code>	Draw component labels and observations by ancestral sampling.

11.10 `sim/factor_graph`

A factor graph, the one structure the three problem classes share.

<code>Variable</code>	<i>class</i> . A discrete latent variable.
<code>Factor</code>	<i>class</i> . A non-negative function of a few variables, held as a log table.
<code>FactorGraph</code>	<i>class</i> . Variables, factors, and the bipartite structure between them.
<code>FactorGraph.variable</code>	The variable called <code>name</code> .
<code>FactorGraph.degree</code>	How many factors <code>name</code> appears in.
<code>FactorGraph.neighbours</code>	The factors <code>name</code> appears in, in factor order, once per incidence.
<code>FactorGraph.is_tree</code>	Whether the bipartite graph is acyclic and connected.
<code>FactorGraph.log_density</code>	<code>sum_f log psi_i f</code> at one full assignment: the unnormalized log-density.
<code>FactorGraph.forney</code>	The normal (Forney) form: every variable of degree above two passes through an equality factor, so each variable is an edge between exactly two factors.
<code>from_potts</code>	The Potts model of <code>snakes_and_ladders.likelihood.potts.log_weights</code>
<code>from_hmm</code>	A hidden Markov chain with its observations folded in.
<code>from_tree</code>	One site of an alignment on a tree: pruning's own factor graph.
<code>from_coupled</code>	The coupled spatio-sequential model of issue #290.
<code>from_parity_check</code>	A parity-check code with its channel output folded in (<code>sec:ldpc</code>).
<code>from_trellis</code>	A terminated convolutional code with its channel output folded in.

11.11 `sim/fixtures`

The registry of supported problem instances (issue #382).

Fixture	<i>class</i> . One loaded fixture: the instance, and what is known about it.
path_of	The file a problem's tier is declared in.
fixture	Load the instance a problem declares at one tier.
problems	Every problem the registry carries, by name.
tiers	The tiers one problem declares an instance at, smallest first.
fixtures	Every problem's instance at one tier, in problem order.
declared	The fixture file's fields as written, for a caller reading one field.
StaleBaselineError	<i>class</i> . Raised when a record's numbers are not the ones this tree produces.
Measurement	<i>class</i> . One number a reference algorithm achieved, with what produced it.
Baseline	<i>class</i> . One fixture's baseline record: the numbers, and what they were computed from.
Baseline.measurement	The measurement named name .
Baseline.value	The scalar named name .
Baseline.values	The vector named name .
baseline_path	Where a problem's tier records its baseline.
read_baseline	Parse a baseline record without checking it against this machine.
write_baseline	Write record to its own path, as the JSON read_baseline reads.
baseline	The baseline record for a problem's tier, refused if it has gone stale.
baselines	Every (problem , tier) that carries a record, in problem order.

11.12 sim/galois

GF(2^m) arithmetic: the field Reed–Solomon needs and nothing here had.

BinaryField	<i>class</i> . GF(2^m), as its exponential and logarithm tables.
BinaryField.order	The number of elements, $2 ** m$.
BinaryField.nonzero	The multiplicative group's order, $2 ** m - 1$.
BinaryField.multiply	left * right , through the logarithm tables.
BinaryField.inverse	value ** -1.
BinaryField.divide	left / right .
BinaryField.power	value ** exponent for any integer exponent, including negative.
BinaryField.alpha	alpha ** exponent , the generator raised to a power.
field	GF($2 ** m$), built once and cached.
polynomial_multiply	Multiply two polynomials over the field, coefficients low degree first.
polynomial_evaluate	Horner's rule over the field, coefficients low degree first.
polynomial_remainder	dividend mod divisor over the field, coefficients low degree first.
bits_from_symbols	Each symbol as m bits, most significant first, concatenated.
symbols_from_bits	The inverse of bits_from_symbols .

11.13 sim/graph

Undirected graphs for the Potts model, and N-D lattice constructors.

<code>BoundaryCondition</code>	<i>class</i> . How a lattice's outermost sites connect.
<code>PottsGraph</code>	<i>class</i> . An undirected graph carrying a per-edge Potts coupling.
<code>PottsGraph.edge_index</code>	The edges as an <code>(n_edges, 2)</code> array of node indices, derived once.
<code>PottsGraph.edge_coupling</code>	The couplings as a <code>float64</code> array, in the graph's edge order.
<code>PottsGraph.weighted_edges</code>	Yield each edge with the coupling on it, in the graph's edge order.
<code>PottsGraph.compressed_adjacency</code>	The adjacency in compressed-row form: offsets, neighbours, couplings.
<code>PottsGraph.incidence</code>	The compressed adjacency, derived once per graph and then reused.
<code>PottsGraph.endpoints</code>	The edges as three arrays in edge order: first, second, coupling.
<code>PottsGraph.to_rustworkx</code>	This graph as a <code>rustworkx.PyGraph</code> : one node per site, the coupling as edge data.
<code>PottsGraph.from_rustworkx</code>	A <code>PottsGraph</code> from a <code>PyGraph</code> whose edge data are couplings.
<code>PottsGraph.is_open_chain_lattice_graph</code>	Whether this graph is a 1-D lattice with an open boundary. Build an N-D lattice as a <code>PottsGraph</code> , with a uniform coupling.
<code>erdos_renyi_graph</code>	A $G(n, p)$ random graph as a <code>PottsGraph</code> , with a uniform coupling.
<code>triangular_lattice_graph</code>	A 2-D triangular lattice: the square lattice plus one diagonal per cell.

11.14 sim/gtr

The general time-reversible substitution model, and its reduction to JC.

<code>n_exchangeabilities</code>	Number of free entries in a k-state symmetric exchangeability matrix.
<code>exchangeability_matrix</code>	Fill a symmetric <code>(k, k)</code> matrix from its upper triangle.
<code>gtr_rate_matrix</code>	The normalized GTR rate matrix for these exchangeabilities and <code>pi</code> .
<code>exchangeabilities_from_free</code>	Complete a free parameter vector to a full exchangeability vector.
<code>reversible_transition_probabilities</code>	$P(t) = \exp(Q \ t)$ for a reversible <code>Q</code> , by symmetric eigendecomposition.

11.15 sim/hmm

Ground truth and data generation for the discrete-HMM reference instance.

<code>HmmParams</code>	<i>class</i> . Fully-specified truth for an HMM fixture.
<code>HmmParams.segment_lengths</code>	One length per chain.
<code>HmmParams.n_sequences</code>	How many chains the batch holds.
<code>HmmParams.rectangular</code>	Whether every chain is the same length.
<code>HmmParams.sequence_length</code>	The length every chain shares.

<code>HmmParams.emission</code>	The emission matrix, for a fixture whose emissions are categorical.
<code>HmmParams.n_symbols</code>	Emission alphabet size, for a categorical fixture.
<code>HmmParams.from_declared</code>	Build the truth from an HMM fixture's declared mapping.
<code>SimulatedHmmDataset</code>	<i>class</i> . A simulated HMM dataset together with the hidden path and its truth.
<code>SimulatedHmmDataset.rectangular</code>	Whether every chain is the same length, and so whether the arrays are 2-D.
<code>SimulatedHmmDataset.batch</code>	The observations as segments, whatever shape they are stored in.
<code>SimulatedHmmDataset.emission</code>	The emission matrix, for a categorical dataset.
<code>simulate_sequences</code>	Draw hidden state paths and observation sequences by ancestral sampling.

11.16 `sim/jc`

Closed-form k-state Jukes-Cantor model.

<code>jc_transition_probabilities</code>	Closed-form k-state Jukes-Cantor transition probabilities $P(t)$.
<code>jc_rate_matrix</code>	The k-state Jukes-Cantor rate matrix Q underlying <code>jc_transition_probabilities</code> .

11.17 `sim/ldpc`

Low-density parity-check codes: two constructions, the channels, an encoder.

<code>ParityCheck</code>	<i>class</i> . A sparse parity-check matrix as offsets into one edge array.
<code>ParityCheck.from_edges</code>	Build from the nonzeros (<code>variables[e]</code> , <code>checks[e]</code>) in any order.
<code>ParityCheck.from_dense</code>	Build from a dense (<code>n_checks</code> , <code>n_bits</code>) 0/1 array.
<code>ParityCheck.n_edges</code>	The nonzeros of H .
<code>ParityCheck.column_weights</code>	Ones per column: the degree of each bit.
<code>ParityCheck.row_weights</code>	Ones per row: the degree of each check.
<code>ParityCheck.dense</code>	H as a (<code>n_checks</code> , <code>n_bits</code>) <code>uint8</code> array, for small codes.
<code>ParityCheck.syndrome</code>	Hc over $GF(2)$: zero on every check exactly when <code>bits</code> is a codeword.
<code>gallager_code</code>	One member of Gallager's regular (<code>column_weight</code> , <code>row_weight</code>) ensemble.
<code>bicycle_code</code>	MacKay, Mitchison and McFadden's (2004) bicycle construction.
<code>Channel</code>	<i>class</i> . A memoryless binary-input channel, seen only through its LLRs.
<code>Channel.log_likelihood_ratios</code>	Transmit codeword and return <code>eq:ldpc-llr</code> per bit.
<code>BinarySymmetricChannel</code>	<i>class</i> . Each bit is flipped independently with probability <code>flip_probability</code> .
<code>BinarySymmetricChannel.log_likelihood_ratios</code>	<i>No docstring</i> .

BinaryErasureChannel	<i>class.</i> Each bit is erased independently with probability <code>erasure_probability</code> .
BinaryErasureChannel.log_likelihood_ratios	<i>No docstring.</i>
BinaryInputGaussianChannel	<i>class.</i> Antipodal signalling $x = 1 - 2c$ plus $N(0, \sigma^2)$ noise.
BinaryInputGaussianChannel.log_likelihood_ratios	<i>No docstring.</i>
all_zero_transmission	The LLRs of the all-zero codeword sent through <code>channel</code> .
gf2_rank	The rank of a 0/1 array over GF(2).
gf2_inverse	The inverse of a square 0/1 array over GF(2).
null_space	A basis of $\{x : \text{matrix } x = 0\}$ over GF(2), as rows.
generator_matrix	A (k, n) basis of the code, systematic in the non-pivot columns.
encode	The codeword carrying <code>message</code> in the code's free positions.
ChannelParams	<i>class.</i> The three channels a code fixture declares.
ChannelParams.symmetric_channel	The declared binary symmetric channel.
ChannelParams.erasure_channel	The declared binary erasure channel.
ChannelParams.gaussian_channel	The declared binary-input Gaussian channel.
LdpcParams	<i>class.</i> Fully-specified truth for a Gallager-ensemble fixture.
LdpcParams.code	Draw the ensemble member this fixture declares.
LdpcParams.from_declared	Build the truth from an LDPC fixture's declared mapping.
BicycleParams	<i>class.</i> Fully-specified truth for a bicycle-code fixture.
BicycleParams.code	Draw the bicycle code this fixture declares.
BicycleParams.from_declared	Build the truth from a bicycle-code fixture's declared mapping.

11.18 sim/mixture

Ground truth and data generation for a finite Gaussian mixture.

BinInstance	<i>class.</i> One declared instance of a fixture: a decimation factor and its tier.
MixtureParams	<i>class.</i> Fully-specified truth for a Gaussian mixture fixture.
MixtureParams.n_components	Components in the mixture.
MixtureParams.n_channels	Entries an observation carries; 1 for a scalar observation.
MixtureParams.at	The declared instance carrying <code>marker</code> , at its own sample count.
MixtureParams.from_declared	Build the truth from a Gaussian-mixture fixture's declared mapping.
SimulatedMixtureDataset	<i>class.</i> A simulated mixture dataset together with the labels and its truth.
simulate_mixture	Draw component labels and observations by ancestral sampling.

11.19 sim/newick

Newick I/O for rooted binary trees: the package's single source of Newick functionality (root CLAUDE.md, "Package Surface").

<code>count_topologies</code>	Count distinct rooted binary tree topologies for <code>n_taxa</code> leaves.
<code>to_newick</code>	Serialize a tree to Newick format.
<code>validate_newick</code>	Check whether <code>s</code> is a well-formed Newick string for a rooted binary tree.
<code>validate_unrooted_newick</code>	Check whether <code>s</code> is well-formed Newick for the trifurcating-root unrooted convention.

11.20 sim/params

Typed loader for the tree fixture files.

<code>SimulationParams</code>	<i>class.</i> Fully-specified inputs for <code>snakes_and_ladders.sim.simulate.simulate_alignment</code> .
<code>SimulationParams.from_declared</code>	Build the truth from a tree fixture's declared mapping.

11.21 sim/polar

Polar codes: the transform, the frozen set it earns, and the Reed-Muller rule.

<code>polar_transform</code>	$F^{\{(x) \ n\}}$: the $2^n \times 2^n$ generator over $GF(2)$.
<code>bec_reliability</code>	Bhattacharyya Z per synthetic channel on the erasure channel: exact.
<code>gaussian_reliability</code>	Bhattacharyya Z per synthetic channel on the Gaussian channel: approximate.
<code>polar_information_set</code>	The <code>n_info</code> most reliable bit-channels, ascending.
<code>reed_muller_information_set</code>	$RM(\text{order}, n)$'s information set: the rows of weight $\geq 2^{(n - \text{order})}$.
<code>PolarCode</code>	<i>class.</i> $N = 2^n$ bits, k of them carrying the message.
<code>PolarCode.n_bits</code>	The block length $N = 2^n$.
<code>PolarCode.n_info</code>	The message length k .
<code>PolarCode.rate</code>	k / N .
<code>PolarCode.frozen</code>	The bit-channels held to zero, ascending.
<code>PolarCode.encode</code>	$u \in G$, with the message on the information channels and zeros elsewhere.
<code>PolarCode.generator</code>	The $k \times N$ generator: the transform's information rows.
<code>parity_check</code>	The code as a <code>ParityCheck</code> .
<code>bec_polar_code</code>	The code the exact erasure recursion picks at this length and rate.
<code>gaussian_polar_code</code>	The code the Gaussian approximation picks at this length and rate.
<code>reed_muller_code</code>	$RM(\text{order}, n_stages)$ as a <code>PolarCode</code> : the same transform, the weight rule.
<code>capacity</code>	$I(W_i) = 1 - Z(W_i)$, which holds <i>exactly</i> on the erasure channel only.
<code>PolarParams</code>	<i>class.</i> Fully-specified truth for a polar fixture.
<code>PolarParams.code</code>	Build the code this fixture declares.

`PolarParams.from_declared` Read a polar fixture.

11.22 `sim/potts`

Spin configurations for a k -state Potts model on a general graph.

<code>site_field</code>	field as one row per site, whichever of the two shapes it arrives in.
<code>energies</code>	$E(s) = -\sum_i h_i[s_i] - \sum_{(ij)} J_{ij} [s_i == s_j]$, per configuration.
<code>energy</code>	$E(s)$ of one labelling, for any state count: <code>energies</code> at <code>n = 1</code> .
<code>PottsMetrics</code>	<i>class</i> . What a Potts labelling means: its energy, from <code>energy</code> (issue #778).
<code>heat_bath_log_weights</code>	One site's unnormalized log conditional: its own field plus its neighbours'.
<code>owner_rows</code>	Which site each entry of the compressed adjacency belongs to.
<code>local_fields</code>	Every site's unnormalized log conditional at once.
<code>critical_coupling</code>	The exact q -state Potts transition on a square lattice, $\ln(1 + \sqrt{q})$.
<code>PottsLatticeParams</code>	<i>class</i> . Fully-specified truth for a Potts-lattice fixture.
<code>PottsLatticeParams.from_declared</code>	Build the truth from a Potts-lattice fixture's declared mapping.
<code>SimulatedPottsDataset</code>	<i>class</i> . Simulated Potts configurations together with the graph and truth.
<code>simulate_potts</code>	Draw <code>n_samples</code> Potts configurations on <code>graph</code> .
<code>SpatioOnlyParams</code>	<i>class</i> . A Potts prior over class labels whose field is driven by a per-site covariate.
<code>SpatioOnlyParams.from_declared</code>	Build the truth from a per-site-field Potts fixture's declared mapping.
<code>spatio_only_field</code>	$\alpha[m] * \log(\text{size}[n] / \text{size_bar})$ as one row per site.

11.23 `sim/potts_chain`

The Potts chain in a field as a declared instance: its parameters, its loader, its exact draw.

<code>PottsParams</code>	<i>class</i> . Fully-specified truth for a Potts-chain fixture.
<code>PottsParams.from_declared</code>	Build the truth from a Potts fixture's declared mapping.
<code>simulate_chains</code>	Draw <code>n_chains</code> exact samples from the truth in <code>params</code> .

11.24 `sim/reed_solomon`

Reed–Solomon: the first decoder here that is algebraic rather than probabilistic.

<code>ReedSolomon</code>	<i>class</i> . An $RS(n, k)$ code over $GF(2^m)$, $n = 2^m - 1$.
<code>ReedSolomon.field</code>	The field the code is over.
<code>ReedSolomon.n_parity</code>	$n - k$, the number of parity symbols.
<code>ReedSolomon.correctable</code>	$t = (n - k) // 2$, the guaranteed symbol-error capacity.
<code>ReedSolomon.minimum_distance</code>	$n - k + 1$: the Singleton bound, met with equality.

<code>reed_solomon</code>	The $RS(2^m - 1, k)$ code whose generator has roots $\alpha^1 \dots \alpha^{(n-k)}$.
<code>encode</code>	Systematically: the message, then the remainder that makes it divisible.

11.25 `sim/simulate`

Pre-order sequence simulation under the k-state Jukes-Cantor model.

<code>SimulatedDataset</code>	<i>class</i> . A simulated alignment together with the parameters that generated it.
<code>simulate_alignment</code>	Simulate an alignment under the k-state Jukes-Cantor model.

11.26 `sim/simulator`

One way to simulate a problem: `(params, rng) -> dataset`, registered beside its loader.

<code>Simulator</code>	<i>class</i> . Data drawn from a declared model under a declared generator, with the truth beside it.
<code>simulate_tree</code>	<code>simulate_alignment</code> on the fixture's tree, states, root and sites.
<code>simulate_potts_lattice</code>	<code>simulate_potts</code> on the lattice the fixture declares.
<code>simulate_spatio_only</code>	<code>simulate_potts</code> on the fixture's own graph and field.
<code>simulate_potts_chain</code>	The exact open-chain draw, as <code>snakes_and_ladders.opt.potts.simulate_chains</code> makes it.

11.27 `sim/spatio_sequential`

The coupled spatio-sequential model of issue #290: parameters and simulator.

<code>circulant_transition</code>	The circulant transition: t on the diagonal, the rest spread evenly.
<code>SpatioSequentialParams</code>	<i>class</i> . Fully-specified truth for the coupled model.
<code>SpatioSequentialParams. segment_lengths</code>	The chain's segments, which is one whole chain where none is declared.
<code>SpatioSequentialParams. transition</code>	The circulant transition A_m of <code>eq:joint</code> , the same for every class.
<code>SpatioSequentialParams. scaled_graph</code>	The graph with $\beta * J$ as couplings: the prior at temperature one.
<code>SpatioSequentialParams. from_declared</code>	Load and validate a coupled spatio-sequential fixture yaml. The spatial graph is declared as a lattice — extent, boundary and a uniform coupling — rather than as an edge list, because that is the family the model's enumerable instances come from and an edge list would make the file unreadable at the sizes the oracle reaches.
<code>SimulatedSpatioSequential</code>	<i>class</i> . One draw from the model.
<code>simulate_spatio_sequential</code>	Draw labels, chains and observations under one generator.
<code>gated_log_density</code>	$\log P(x_{\{sn\}} \mid k, \theta_{a,m})$ for every node, position, class and state.
<code>coupled_factor_graph</code>	The model on observations as the factor graph of <code>eq:joint</code> .

`canonical_spatio_sequential` The instance small enough to enumerate: a 2x2 open lattice, $M = K = 2, S = 6$.

11.28 `sim/topology`

NNI and SPR neighbourhood generators over unrooted binary topologies.

<code>MoveSet</code>	<i>class</i> . Which neighbourhood the search proposes from.
<code>Model</code>	<i>class</i> . Which substitution model the continuous fit uses.
<code>enumerate_topologies</code>	Every unrooted binary topology on <code>leaf_names</code> , each exactly once.
<code>random_topology</code>	Draw an unrooted binary topology on <code>leaf_names</code> , uniformly by construction.
<code>leaf_bipartitions</code>	The set of leaf-set bipartitions induced by every edge of <code>topology</code> .
<code>branch_splits</code>	The split each branch induces, in <code>pruning_torch.branch_order</code> order.
<code>nni_neighbours</code>	Generate the nearest-neighbour-interchange neighbourhood of <code>topology</code> .
<code>spr_neighbours</code>	Generate the subtree-prune-and-regraft neighbourhood of <code>topology</code> .
<code>robinson_foulds</code>	Robinson-Foulds distance: splits in one topology and not the other.
<code>normalized_robinson_foulds</code>	Robinson-Foulds distance scaled to <code>[0, 1]</code> .

11.29 `sim/tree`

Minimal tree representation for the simulator.

<code>Node</code>	<i>class</i> . A node in a rooted tree.
<code>Node.is_leaf</code>	Whether this node has no children.
<code>preorder</code>	Yield every node in the tree rooted at <code>root</code> , parent before children.
<code>edges</code>	Yield every (<code>parent</code> , <code>child</code>) edge in the tree rooted at <code>root</code> .
<code>balanced_tree</code>	A balanced topology on <code>n_taxa</code> leaves whose root-to-tip paths are <code>height</code> .